BÀI TẬP BUỔI 2 — LÊ SƠN TRƯỜNG

D2L 1.0.3 — Exercises chương 1–5

162 câu cấp chính, 28 nhóm Exercises; có lời giải các ý con.
Đã chạy thí nghiệm bằng PyTorch, seed42, và lưu output notebook.

Phần còn cần thao tác: nộp CSV Kaggle để có điểm leaderboard; đo GPU bằng môi trường CUDA nếu bắt buộc; upload hồ sơ lên Drive.
Không giả lập các ảnh màn hình tài khoản. Đây là bản xuất lời giải và kết quả thực nghiệm.

Dữ liệu: Abalone UCI, Fashion-MNIST chính thức, House Prices của D2L.
Xem README.md để chạy lại, coverage.csv để đối chiếu từng mục.

Lê Sơn Trường — Chương 1, mục 1.9

1.9 — Exercises

Câu 1 — Phần nào của chương trình có thể học?

Trong bài phân loại hai nhóm điểm, việc viết thủ công đường chia hoặc ngưỡng
tọa độ là một heuristic. Có thể thay bằng MLP học từ các cặp (x1, x2, nhãn).
Trọng số được tối ưu theo loss thay vì người lập trình chọn từng ngưỡng.
Ví dụ khác: xếp hạng tìm kiếm có thể học từ truy vấn, kết quả hiển thị và phản hồi;
phản hồi phải được xử lý thiên lệch vị trí, không coi mọi click là nhãn hoàn hảo.

Câu 2 — Nhiều ví dụ nhưng khó viết quy tắc

Nhận dạng chữ viết tay, phân loại ảnh quần áo, chuyển giọng nói thành văn bản.
Ta có thể cung cấp nhiều ví dụ đầu vào/đáp án, nhưng khó viết đầy đủ quy tắc
cho mọi nét chữ, góc chụp hay giọng nói. Deep learning phù hợp khi dữ liệu và
ngân sách tính toán đủ; không phải mọi bài toán đều cần mạng sâu.

Câu 3 — Thuật toán, dữ liệu và tính toán

Dữ liệu cung cấp thông tin; thuật toán quyết định cách biểu diễn và học;
tính toán giới hạn kích thước mô hình, số thí nghiệm và tốc độ cập nhật.
Dữ liệu ít/dạng bảng thường nên có baseline tuyến tính; ảnh lớn có thể cần CNN.
Nhãn nhiễu hoặc dữ liệu không đại diện không được sửa chỉ bằng GPU mạnh hơn.
Chọn thuật toán phải xét số mẫu, số chiều, kiểu dữ liệu, RAM/VRAM và thời gian.

Câu 4 — Nơi có thể thử end-to-end

Ví dụ một hệ thống đọc hóa đơn đang ghép OCR, nhận diện trường và luật kiểm tra;
có thể thử học trực tiếp ảnh → các trường cần trích xuất. Một dây chuyền kiểm tra
sản phẩm có thể học ảnh → loại lỗi thay cho đặc trưng thủ công + ngưỡng.
Đây là các tình huống minh họa, không khẳng định end-to-end luôn tốt hơn:
cần dữ liệu có nhãn phù hợp, tiêu chí đánh giá và khả năng kiểm tra lỗi của từng bước.

Lê Sơn Trường — Chương 2, mục 2.1

2.1 — Data Manipulation (2 câu)

Câu 1

X < Y và X > Y so sánh từng phần tử, trả về tensor torch.bool cùng shape sau broadcasting. Không trả về một boolean chung cho hai tensor. Notebook chạy các thao tác tạo, reshape, indexing, nối, toán tử và so sánh của mục này.

Câu 2

Ví dụ (2,1,4) + (1,3,1) cho shape (2,3,4). So từ chiều cuối: hai kích thước phải bằng nhau hoặc một bên bằng 1. (2,3,4) + (2,2,4) không tương thích ở chiều giữa nên báo lỗi. Không tạo bản sao vật lý của mọi phần tử chỉ để broadcast.

Ghi nhận chạy thật — mục 2.1

Broadcast (2, 1, 4) + (1, 3, 1) -> (2, 3, 4): [[[0, 1, 2, 3], [1, 2, 3, 4], [2, 3, 4, 5]], [[4, 5, 6, 7], [5, 6, 7, 8], [6, 7, 8, 9]]]

Index X[1,2]=6.0; sum=66.0; NumPy round-trip shape=(3, 4)

Slice assignment then in-place +=: [[14.0, 13.0, 16.0, 15.0], [13.0, 14.0, 15.0, 16.0], [12.0, 12.0, 12.0, 12.0]]; id preserved=True

Tensor scalar conversions: item=3.5, float=3.5

Kết quả mục 2.1 — tensor_operations

operation
result
X
[[0.0, 1.0, 2.0, 3.0], [4.0, 5.0, 6.0, 7.0], [8.0, 9.0, 10.0, 11.0]]
Y
[[2.0, 1.0, 4.0, 3.0], [1.0, 2.0, 3.0, 4.0], [4.0, 3.0, 2.0, 1.0]]
X+Y
[[2.0, 2.0, 6.0, 6.0], [5.0, 7.0, 9.0, 11.0], [12.0, 12.0, 12.0, 12.0]]
X-Y
[[-2.0, 0.0, -2.0, 0.0], [3.0, 3.0, 3.0, 3.0], [4.0, 6.0, 8.0, 10.0]]
X*Y
[[0.0, 1.0, 8.0, 9.0], [4.0, 10.0, 18.0, 28.0], [32.0, 27.0, 20.0, 11.0]]
X/Y
[[0.0, 1.0, 0.5, 1.0], [4.0, 2.5, 2.0, 1.75], [2.0, 3.0, 5.0, 11.0]]
XY
[[0.0, 1.0, 16.0, 27.0], [4.0, 25.0, 216.0, 2401.0], [4096.0, 729.0, 100.0, 11.0]]
exp(X) [[1.0, 2.7182817459106445, 7.389056205749512, 20.08553695678711], [54.598148345947266, 148.4131622314453, 403.4288024902344, 1096.6331787109375], [2980.9580078125, 8103.083984375, 22026.46484375, 59874.140625]]
X[-1]
[8.0, 9.0, 10.0, 11.0]
X[1:3]
[[4.0, 5.0, 6.0, 7.0], [8.0, 9.0, 10.0, 11.0]]
X==Y
[[False, True, False, True], [False, False, False, False], [False, False, False, False]]
X<Y
[[True, False, True, False], [False, False, False, False], [False, False, False, False]]
X>Y
[[False, False, False, False], [True, True, True, True], [True, True, True, True]]
cat rows [[0.0, 1.0, 2.0, 3.0], [4.0, 5.0, 6.0, 7.0], [8.0, 9.0, 10.0, 11.0], [2.0, 1.0, 4.0, 3.0], [1.0, 2.0, 3.0, 4.0], [4.0, 3.0, 2.0, 1.0]]
cat columns [[0.0, 1.0, 2.0, 3.0, 2.0, 1.0, 4.0, 3.0], [4.0, 5.0, 6.0, 7.0, 1.0, 2.0, 3.0, 4.0], [8.0, 9.0, 10.0, 11.0, 4.0, 3.0, 2.0, 1.0]]

Lê Sơn Trường — Chương 2, mục 2.2

2.2 — Data Preprocessing (5 câu)

Câu 1

Tải Abalone thật từ UCI, đếm ô thiếu bằng df.isna().mean() và xác định kiểu cột. Bộ này có 4.177 hàng, 9 cột: Sex là categorical; 7 số đo liên tục và Rings là số nguyên. Xét toàn bộ bảng: 8/9 cột số, 1/9 categorical, không có cột văn bản tự do; xét riêng 8 đặc trưng: 7/8 số và 1/8 categorical. Dữ liệu gốc không có giá trị thiếu theo kiểm tra parser, nhưng không suy ra mọi số đo đều hợp lý.

Câu 2

df.loc[:, ['Sex','Length','Rings']] hoặc df[['Length','Rings']] chọn theo tên.
df.loc[df['Sex']=='F', ['Length','Rings']] lọc hàng và cột theo điều kiện.
Tên cột giúp code ít phụ thuộc thứ tự hơn iloc.

Câu 3

Đo memory_usage(deep=True).sum() và thời gian read_csv trên dữ liệu lặp tăng dần trong notebook; không cố nạp đến khi máy hết RAM. Giới hạn thực tế gồm RAM, bản sao trung gian, object/string overhead, tốc độ đĩa và parser. CSV thường tốn RAM nhiều hơn kích thước file. Server nhiều RAM chỉ nâng giới hạn, không bỏ giới hạn. Với bảng lớn, đọc chunksize, chỉ usecols, chọn dtype, dùng Parquet/Arrow hoặc xử lý ngoài RAM. Chưa chạy trên server riêng nên không trình bày số đo server như kết quả thực nghiệm.

Câu 4

One-hot với quá nhiều category gây bảng rất rộng. Có thể dùng sparse one-hot, hashing hoặc embedding, gộp nhóm hiếm. ID duy nhất thường nên bỏ nếu không có ý nghĩa dự đoán; mạng dễ ghi nhớ. Target encoding phải fit trong fold train để tránh leakage.

Câu 5

NumPy với save/load phù hợp mảng đồng nhất; Pillow đọc ảnh; PyArrow/Polars xử lý bảng; SQLite/DuckDB cho truy vấn đĩa. Notebook thực hiện vòng lưu/đọc .npy và một ảnh PNG bằng Pillow. Chọn công cụ theo cấu trúc dữ liệu, không coi chúng thay thế hoàn toàn nhau.

Ghi nhận chạy thật — mục 2.2

Abalone: (4177, 9), missing cells=0; NumPy/Pillow round-trips verified.

Kết quả mục 2.2 — abalone_head

Sex	Length	Diameter	Height	WholeWeight	ShuckedWeight	VisceraWeight	ShellWeight	Rings
M	0.455	0.365	0.095	0.514	0.2245	0.101	0.15	15
M	0.35	0.265	0.09	0.2255	0.0995	0.0485	0.07	7
F	0.53	0.42	0.135	0.677	0.2565	0.1415	0.21	9
M	0.44	0.365	0.125	0.516	0.2155	0.114	0.155	10
I	0.33	0.255	0.08	0.205	0.0895	0.0395	0.055	7

Kết quả mục 2.2 — abalone_schema

column	dtype	missing_fraction
Sex	object	0
Length	float64	0
Diameter	float64	0
Height	float64	0
WholeWeight	float64	0
ShuckedWeight	float64	0
VisceraWeight	float64	0
ShellWeight	float64	0
Rings	int64	0

Kết quả mục 2.2 — named_selection

Sex	Length	Rings
F	0.53	9
F	0.53	20
F	0.545	16
F	0.55	19
F	0.525	14

Kết quả mục 2.2 — memory_measurements

rows	file_bytes	dataframe_bytes	read_seconds
4177	196142	476310	0.0086224
41770	1960646	4761912	0.0223935
417700	19605686	47617932	0.147277

Lê Sơn Trường — Chương 2, mục 2.3

2.3 — Linear Algebra (12 câu)

Câu 1

$((A^T)^T)ij = (A^T)ji = Aij$, nên $(A^T)^T=A$ cho mọi phần tử.

Câu 2

$(A+B)^Tij=(A+B)ji=Aji+Bji=(A^T+B^T)ij$; hai ma trận phải cùng shape.

Câu 3

Với A vuông, $(A+A^T)^T=A^T+(A^T)^T=A^T+A=A+A^T$, nên đối xứng.

Câu 4

X có shape (2,3,4) thì len(X)=2, vì len đo chiều đầu. Dự đoán trước rồi kiểm tra bằng code.

Câu 5

Tensor có ít nhất một chiều: len(X)==X.shape[0]. Tensor scalar (0 chiều)
không có chiều đầu, gọi len sẽ lỗi; vì vậy không nói quy tắc đúng cho mọi tensor.

Câu 6

A trong mục này shape (5,4), A.sum(axis=1) shape (5,); phép chia không
broadcast được với chiều cuối 4. Sửa thành A / A.sum(dim=1,keepdim=True).
Nếu A vuông, code cũ có thể chạy nhưng chia theo cột bởi tổng các hàng, không phải
chuẩn hóa từng hàng. Cần xử lý tổng bằng 0 khi dữ liệu có thể gây trường hợp đó.

Câu 7

Khoảng cách Manhattan là |x1-x2|+|y1-y2| nếu chỉ đi dọc các đường vuông góc.
Không đi chéo xuyên ô phố trong mô hình này; khoảng cách Euclid là cận dưới.

Câu 8

Shape (2,3,4): sum dim 0 → (3,4); dim 1 → (2,4); dim 2 → (2,3).
Nếu keepdim=True, giữ chiều cộng với kích thước 1.

Câu 9

torch.linalg.norm(X) không chỉ định dim/ord với tensor nhiều chiều: căn của
tổng bình phương trị tuyệt đối mọi phần tử (norm Euclid của tensor được làm phẳng).
Chỉ định ord/dim có quy tắc khác; không gọi đây là spectral norm của tensor nhiều chiều.

Câu 10

A: (2^10,2^16), B: (2^16,2^5), C: (2^5,2^14).
Tính (AB)C cần khoảng 2^31+2^29 phép nhân vô hướng; A(BC) cần
2^35+2^40. Trung gian AB có 2^15 phần tử; BC có 2^30 phần tử
(4 GiB FP32 chỉ cho trung gian), nên thứ tự đầu phù hợp hơn.
Notebook đo bản thu nhỏ cùng tỷ lệ, và tính chính xác chi phí/kích thước cho đề gốc;
không tạo trung gian nhiều GiB làm hết RAM.

Câu 11

Lê Sơn Trường — Chương 2, mục 2.3

A: (1024,65536), B: (65536,32), C: (32,65536) nên AB và AC^T có cùng
số phép toán và shape kết quả. Layout contiguous/strides khác có thể khiến kernel
phải pack/copy và thời gian khác; không thể kết luận luôn một bên nhanh hơn.
Nếu C=B.T là view không clone thì C.T có cùng storage/layout với B,
hai phép toán tương đương hơn. Notebook đo thời gian lặp sau warm-up ở kích thước thu nhỏ.

Câu 12

torch.stack([A,B,C],dim=2) cho (100,200,3); stacked[:,:,1] chính là B.
Nếu stack mặc định dim=0 thì shape (3,100,200) và B là stacked[1], không
phải lát cắt ở chiều thứ ba. Cần chọn dim=2 theo yêu cầu đề.

Ghi nhận chạy thật — mục 2.3

Expected broadcasting error: The size of tensor a (4) must match the size of tensor b (5) at non-singleton dimension 1

len(X)=2; norm=65.757126; flattened norm=65.757126

Stack dim=2: (100, 200, 3); recovered B verified

Kết quả mục 2.3 — shapes

operation shape
sum dim 0 (3, 4)
sum dim 1 (2, 4)
sum dim 2 (2, 3)

Kết quả mục 2.3 — row_normalization

```
          0         1         2         3
     0 0.166667 0.333333     0.5
0.181818 0.227273 0.272727 0.318182
0.210526 0.236842 0.263158 0.289474
0.222222 0.240741 0.259259 0.277778
0.228571 0.242857 0.257143 0.271429
```

Kết quả mục 2.3 — original_chain_cost

order	scalar_multiplies	intermediate_float32_bytes
(AB)C	2684354560	131072
A(BC)	1133871366144	4294967296

Kết quả mục 2.3 — chain_benchmark

order	seconds_per_call	scale
(AB)C	0.00015112	1/16 each dimension
A(BC)	0.006307	1/16 each dimension

Kết quả mục 2.3 — layout_benchmark

case	seconds
AB contiguous	0.0006707
AC.T noncontiguous	0.000447
C=B.T view, then A@C.T	0.0006666

Lê Sơn Trường — Chương 2, mục 2.4

2.4 — Calculus (10 câu)

Câu 1

Từ f'(x)=limh→0 (f(x+h)-f(x))/h:

- Hằng c: tử số 0 nên đạo hàm 0.
- x^n, n nguyên dương: khai triển nhị thức, chia h còn nx^(n−1)+O(h);
  giới hạn bằng nx^(n−1). Với n thực, x>0, dùng $x^n=\exp(n \log x)$ và chain rule.
- e^x: thương sai phân $e^x(e^h-1)/h$; từ chuỗi $e^h=1+h+h^2/2!+\ldots$,
  $(e^h-1)/h\to 1$, nên đạo hàm e^x.
- log x, x>0: đặt t=h/x, thương bằng (1/x) log(1+t)/t;
  giới hạn log(1+t)/t=1 (từ log là nghịch đảo exp) nên bằng 1/x.

Câu 2

Quy tắc tổng: tách thương sai phân thành hai thương. Quy tắc tích: thêm/bớt
f(x+h)g(x), được f(x+h)·Δg/h+g(x)·Δf/h; lấy giới hạn cho fg'+gf'.
Quy tắc thương, g(x)≠0: gộp tử f(x+h)g(x)-f(x)g(x+h) rồi chia cho
h g(x+h)g(x); giới hạn là (f'g−fg')/g^2. Các hàm khả vi nên liên tục tại x.

Câu 3

Trong quy tắc tích đặt g=c, g'=0: (cf)'=cf'.

Câu 4

Với x>0, log f=x log x; f'/f=log x+1, nên $(x^x)'=x^x(\log x+1)$.

Câu 5

f'(x)=0 nghĩa là tiếp tuyến ngang/điểm dừng, không tự suy ra cực trị.
$f=x^2$ tại 0 là cực tiểu; $f=x^3$ tại 0 có đạo hàm 0 nhưng không cực trị.

Câu 6

$f(x)=x^3-1/x$, x≠0; f(1)=0 và f'(1)=3+1=4.
Tiếp tuyến tại 1: y=4(x−1). Notebook vẽ miền x>0, không nối qua điểm kỳ dị x=0.

Câu 7

$f=3x_1^2+5\exp(x_2) \Rightarrow$ gradient $[6x_1, 5\exp(x_2)]$.

Câu 8

$f=\|x\|_2 \Rightarrow$ gradient $x/\|x\|_2$ khi x≠0. Tại 0 norm không khả vi;
subgradient là mọi vector norm≤1. Nếu là norm bình phương thì gradient=2x,
kể cả ở 0; phân biệt chỉ số ₂ với lũy thừa ².

Câu 9

∂u/∂a=u_x x_a+u_y y_a+u_z z_a; tương tự với b.
Dạng ma trận: Jacobian của ánh xạ ghép bằng tích các Jacobian theo thứ tự chain rule.

Câu 10

Đạo hàm $f^{-1}(f(x))=x$: $(f^{-1})'(f(x))\, f'(x)=1$.

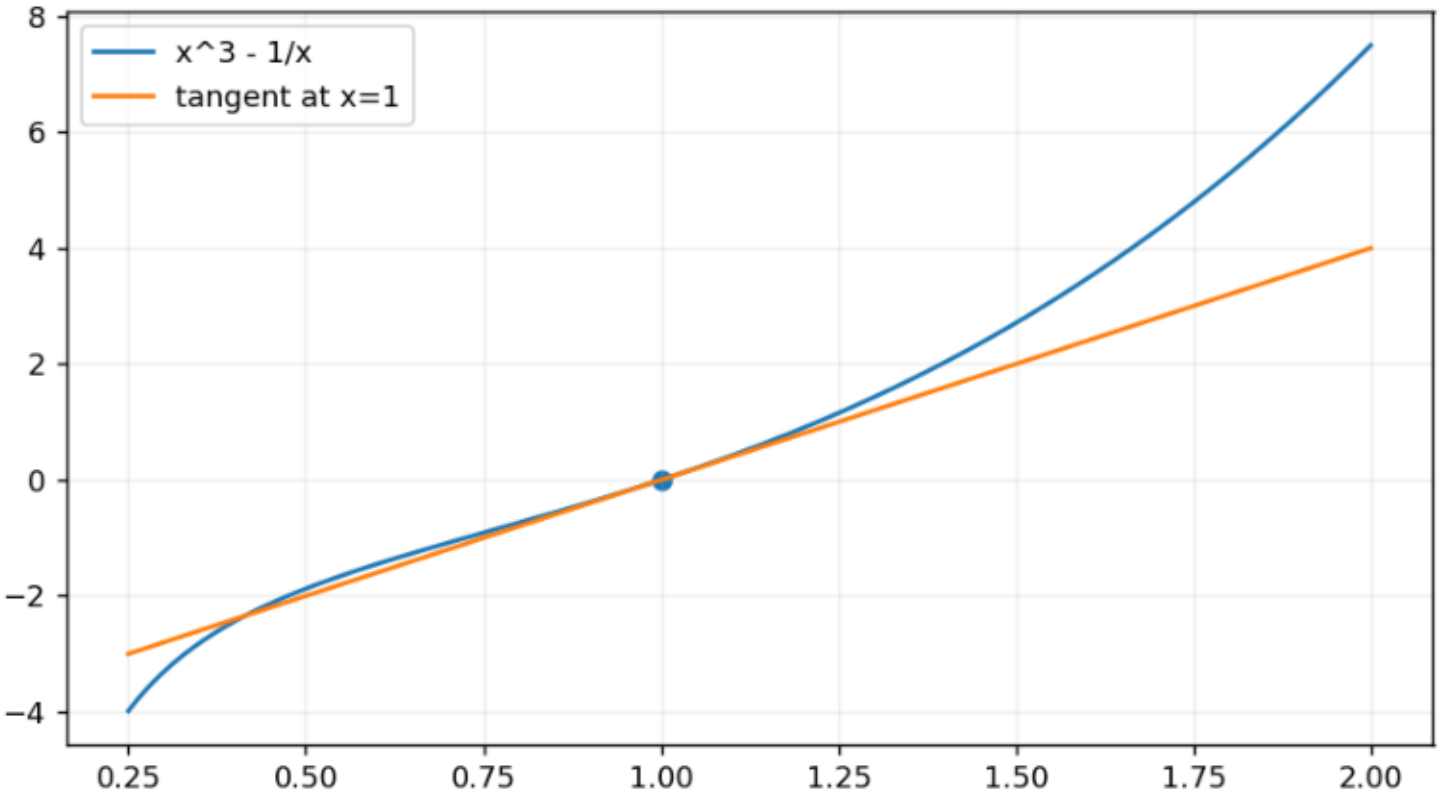
Lê Sơn Trường — Chương 2, mục 2.4

Vì vậy $(f^{-1})'(y)=1/f'(f^{-1}(y))$, khi nghịch đảo khả vi và f' tại đó khác 0.

Ghi nhận chạy thật — mục 2.4

Gradient at (1,2)=[6.0, 36.945281982421875]

Lê Sơn Trường — Kết quả mục 2.4
function_and_tangent



Lê Sơn Trường — Chương 2, mục 2.5

2.5 — Automatic Differentiation (8 câu)

Câu 1

Đạo hàm bậc hai cần giữ/tạo đồ thị cho gradient. Với n tham số, Hessian đầy đủ có n^2 phần tử và cần nhiều phép đạo hàm, nên thường tốn hơn gradient bậc một. Hessian-vector product không cần lưu toàn Hessian và rẻ hơn đáng kể.

Câu 2

PyTorch giải phóng intermediate sau backward mặc định; backward lại cùng graph có thể lỗi. Muốn dùng lại graph cần retain_graph=True ở lần đầu, hoặc tính forward lại. .grad cộng dồn nên phải zero trước bước cập nhật tiếp theo.
Notebook bắt lỗi chủ động rồi minh họa cách làm đúng.

Câu 3

Điều kiện while b.norm()<1000 vẫn là scalar với vector/matrix, nhưng if b.sum()>0 phải viết rõ; if b>0 với nhiều phần tử không có giá trị chân lý duy nhất.
Output không scalar thì backward() cần gradient đầu vào cùng shape, hoặc lấy output.sum().backward(); đây là vector-Jacobian product, không tự cho toàn Jacobian.
Ở một nhánh cố định, f(a)=c·a nên gradient của tổng là c ở mỗi tọa độ; tại ranh giới đổi nhánh phải phân tích riêng.

Câu 4

Notebook tạo x requires_grad, y=sin(x), rồi y.sum().backward(); vẽ y và x.grad. Không dùng cos để tạo đường đạo hàm; cos chỉ có thể dùng làm đối chiếu sau đó.

Câu 5

Đồ thị phụ thuộc: $x \to a=x^2 \to b=\log(a)$; $x \to c=\sin(x)$; $(b,c) \to d=b*c$; $x \to e=1/x$; $(d,e) \to f=d+e$. Miền $x \neq 0$.

Câu 6

Các đạo hàm cục bộ: $a'=2x$; $db/da=1/a$; $c'=\cos x$; $\partial d/\partial b=c$; $\partial d/\partial c=b$; $e'=-1/x^2$; $\partial f/\partial d=\partial f/\partial e=1$.
Suy ra $f'=2\sin(x)/x + \log(x^2)\cos(x) - 1/x^2$.

Câu 7

Forward mode: truyền tangent dx=1 qua từng nút bằng chain rule để được công thức trên.
Reverse mode: bắt đầu adjoint $\bar{f}=1$; $\bar{d}=\bar{e}=1$, $\bar{b}=c$, $\bar{c}=b$, $\bar{a}=c/a$; cộng các nhánh về x: $\bar{x}=2x\cdot c/a + b\cos x - 1/x^2$. Hai cách cho cùng kết quả; notebook so sánh với autograd.

Câu 8

Hàm $R^n \to R^m$: forward thường phù hợp khi n nhỏ, cần khoảng n lượt cho Jacobian; reverse phù hợp m nhỏ, cần khoảng m lượt và lưu intermediate của forward.
Loss mạng thường scalar, n lớn nên reverse mode hiệu quả. Có thể dùng JVP/VJP, checkpointing hoặc vector hóa tùy bộ nhớ và đại lượng cần tính.

Ghi nhận chạy thật — mục 2.5

Expected second backward error: Trying to backward through the graph a second time (or directly access saved tensors after they have already been freed). Saved intermediate values of the graph are freed when you call .backward() or autograd.grad(). Specify retain_graph=True if you need to backward through the graph a second time or if you need to access saved tensors after calling backward.

Repeated backward with retain_graph accumulates grad: 8.0

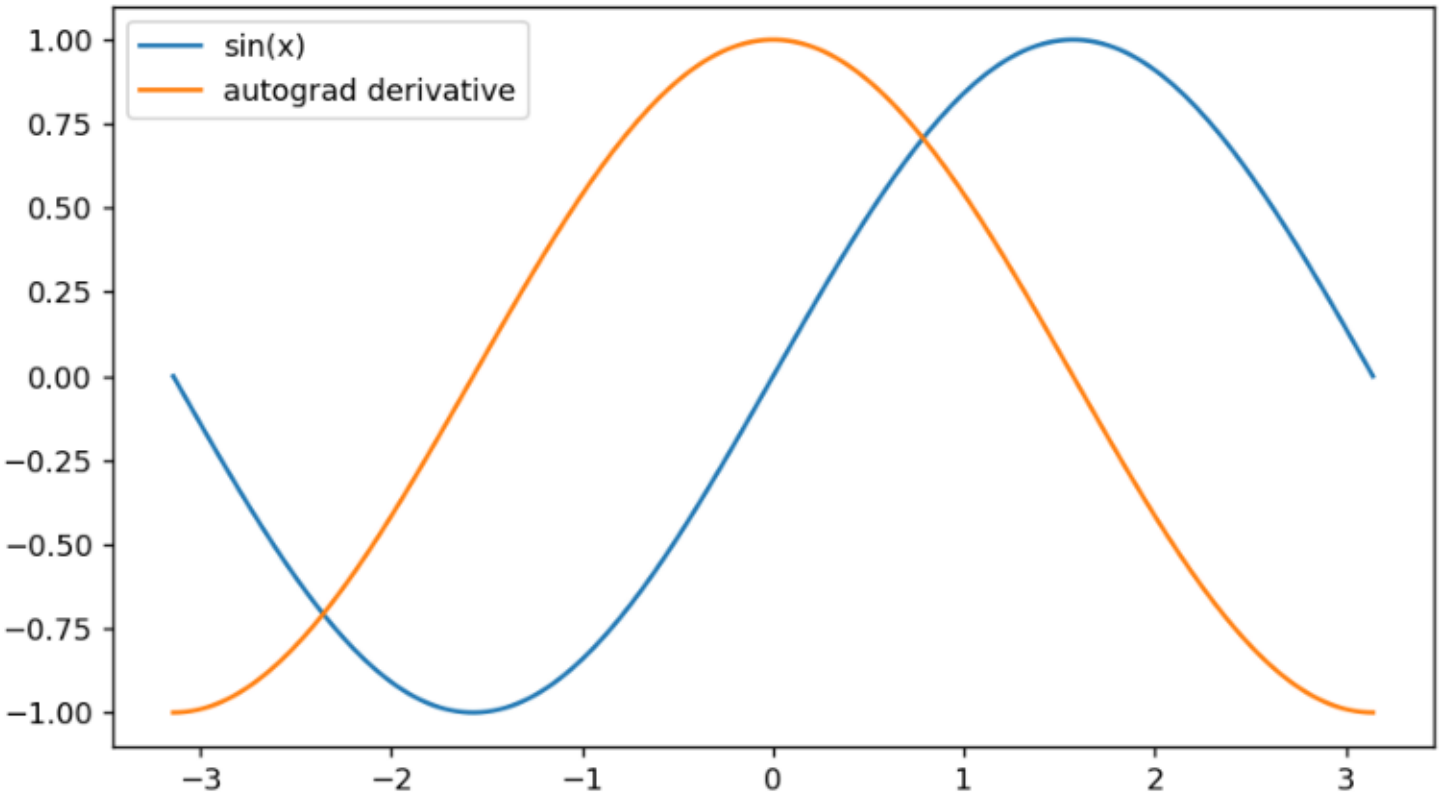
Vector output=[512.0, 1024.0, 1536.0], VJP for sum=[512.0, 512.0, 512.0]

Dependency graph: x -> x^2 -> log(x^2); x -> sin(x); multiply -> +1/x -> f.

Kết quả mục 2.5 — chain_rule

x	autograd	forward_chain_rule	reverse_chain_rule
1.3	1.03105	1.03105	1.03105

Lê Sơn Trường — Kết quả mục 2.5
sin_autograd



Lê Sơn Trường — Chương 2, mục 2.6

2.6 — Probability and Statistics (8 câu)

Câu 1

Với các lần tung đồng xu IID có xác suất p cố định, càng nhiều mẫu càng xác định chính xác p: $\mathrm{Var}(\hat{p})=p(1-p)/n\to 0$. Đây là bất định về tham số, không phải biết chắc kết quả của lần tung tiếp theo.

Câu 2

Ngay khi biết p=0.5 chính xác, một lần tung tiếp theo vẫn ngẫu nhiên; bất định không giảm về 0 chỉ nhờ thêm dữ liệu quá khứ. Phân biệt bất định tham số và nhiễu vốn có.

Câu 3

Với Bernoulli IID: $\mathrm{Var}(\hat{p})=p(1-p)/n$, giảm theo 1/n; độ lệch chuẩn theo $1/\sqrt{n}$. Chebyshev: $P(|\hat{p}-p|\geq\varepsilon)\leq p(1-p)/(n\varepsilon^2)$. CLT cho $\sqrt{n}(\hat{p}-p)\to N(0,p(1-p))$, giúp xấp xỉ xác suất chặt hơn khi điều kiện phù hợp.

Câu 4

Nếu mẫu IID mean 0, variance 1: $\mathrm{Var}(z_m)=1/m$ và mỗi m riêng đều có cận $P(|z_m|\geq\varepsilon)\leq 1/(m\varepsilon^2)$. Nhưng z_m dùng chung các mẫu với z_{m+1}, nên các sự kiện không độc lập; không nhân xác suất/cận như thể độc lập. Nếu mẫu không độc lập, phải thêm covariance, không mặc định Var=1/m.

Câu 5

Đặt a=P(A), b=P(B): $\max(a,b)\leq P(A\cup B)\leq\min(1,a+b)$; $\max(0,a+b-1)\leq P(A\cap B)\leq\min(a,b)$. Từ công thức cộng xác suất và sự bao hàm tập hợp.

Câu 6

Chuỗi Markov A→B→C: $P(A,B,C)=P(A)P(B|A)P(C|B)$ vì C độc lập có điều kiện với A khi biết B.

Câu 7

Đây là phép tính xác suất giả định của bài tập, không phải hướng dẫn xét nghiệm thực tế. Khi H=0: bảng (D1,D2) có (1,1)=0.02, (1,0)=0.08, (0,1)=0.08, (0,0)=0.82. Khi H=1, xác suất cả hai dương là $0.99^2=0.9801$. Với prior 0.0015:

- Sau một dương: $0.99\cdot 0.0015/(0.99\cdot 0.0015+0.1\cdot 0.9985)$.
- Sau hai dương: $0.9801\cdot 0.0015/(0.9801\cdot 0.0015+0.02\cdot 0.9985)$.

Notebook tính giá trị số; không dùng 0.1^2 cho người khỏe vì đề nói hai kết quả có phụ thuộc.

Câu 8

Đây là bài toán toán học danh mục giả định. Với tổng $\alpha_i=1$: $E[\text{return}]=\alpha^T\mu$; $\mathrm{Var}(\text{return})=\alpha^T\Sigma\alpha$. Nếu $\alpha\geq 0$ và chỉ tối đa kỳ vọng, đặt toàn bộ vào tài sản có μ lớn nhất (chia tùy ý nếu đồng hạng). Nếu cho short vô hạn và không ràng buộc rủi ro, bài toán có thể không bị chặn. Có giới hạn rủi ro v: $\max \alpha^T\mu$, subject to $1^T\alpha=1$, $\alpha^T\Sigma\alpha\leq v$ và $\alpha\geq 0$ nếu không cho short. Đề chỉ yêu cầu lập bài toán, không yêu cầu giải quadratic program.

Kết quả mục 2.6 — coin_variance

n	empirical_variance	theoretical_variance
10	0.0244402	0.025
100	0.00260976	0.0025
1000	0.000246974	0.00025
10000	2.54961e-05	2.5e-05

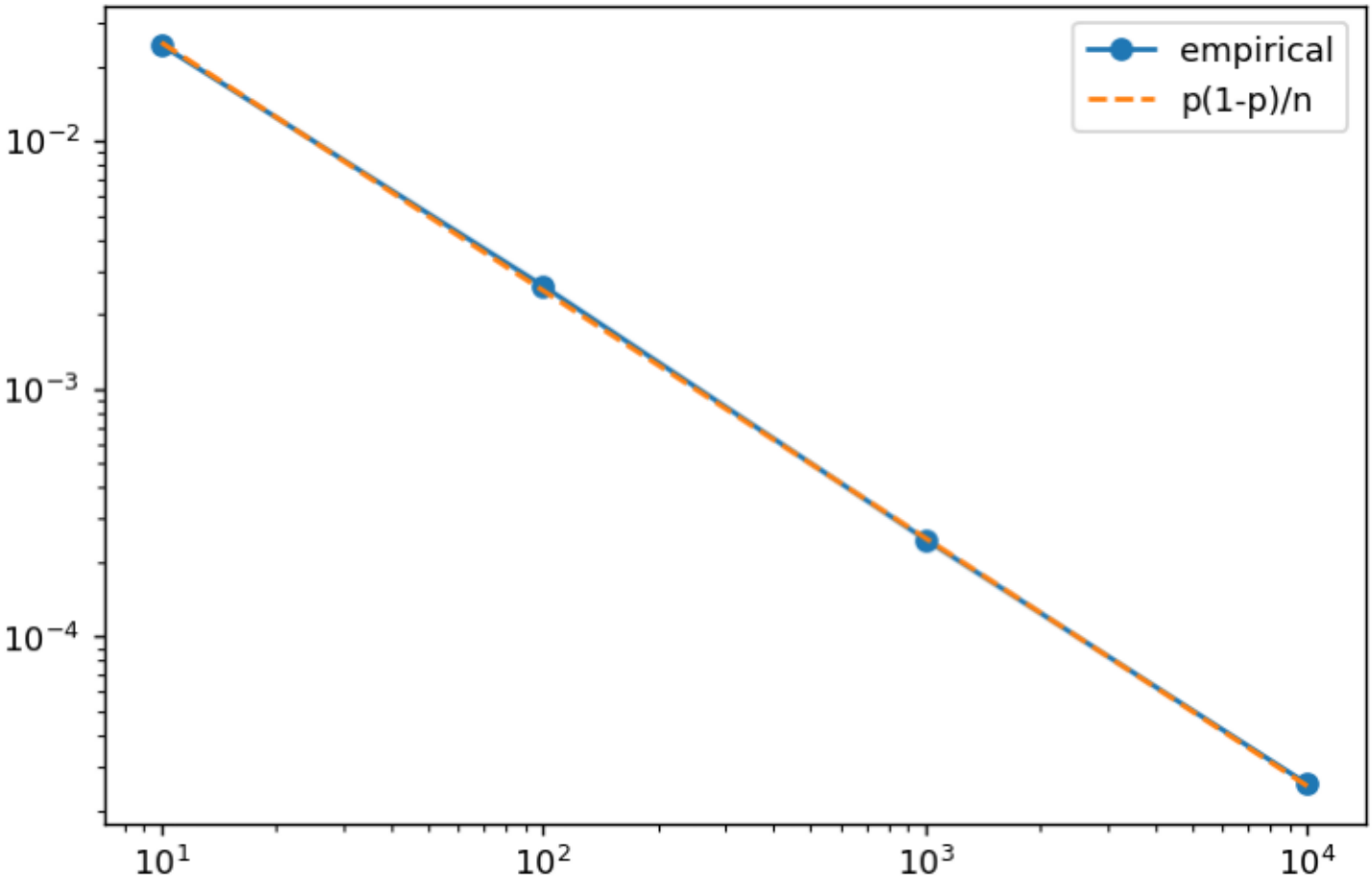
Kết quả mục 2.6 — bayes_exercise

observations	posterior
one positive	0.0146544
two positives, correlated healthy tests	0.0685699

Kết quả mục 2.6 — joint_given_healthy

D1	D2	probability
0	0	0.82
0	1	0.08
1	0	0.08
1	1	0.02

Lê Sơn Trường — Kết quả mục 2.6
variance_vs_n



Lê Sơn Trường — Chương 2, mục 2.7

2.7 — Documentation

PDF không có Exercises đánh số ở mục này. Notebook minh họa help, docstring và chữ ký hàm để luyện tra API, không tự thêm chúng thành câu hỏi của sách.

Ghi nhận chạy thật — mục 2.7

torch.add docstring:

add(input, other, *, alpha=1, out=None) -> Tensor

Adds :attr:other, scaled by :attr:alpha, to :attr:input.

.. math::
    \text{{out}}_i = \text{{input}}_i + \text{{alpha}} \times \text{{other}}_i

Supports :ref:broadcasting to a common shape <broadcasting-semantics>,
:ref:type promotion <type-promotion-doc>, and integer, float, and complex inputs.

Args:
    input (Tensor): the input tensor.
    other (Tensor or Number): the tensor or number to add to :attr:input.

Keyword arguments:
    alpha (Number): the multiplier for :attr:other.
    out (Tensor, optional): the output tensor.

Examples::

    >>> a = torch.randn(4)
    >>> a
    tensor([0.0202, 1.0985, 1.3506, -0.6056])
    >>> torch.add(a, 20)
    tensor([20.0202, 21.0985, 21.3506, 19.3944])

    >>> b = torch.randn(4)
    >>> b
    tensor([-0.9732, -0.3497, 0.6245, 0.4022])
    >>> c = torch.randn(4, 1)
    >>> c
    tensor([[0.3743],
            [-1.7724],
            [-0.5811],
            [-0.8017]])
    >>> torch.add(b, c, alpha=10)
    tensor([[2.7695,   3.3930,   4.3672,   4.1450],
            [-18.6971, -18.0736, -17.0994, -17.3216],
            [-6.7845, -6.1610, -5.1868, -5.4090],
            [-8.9902, -8.3667, -7.3925, -7.6147]])

No numbered Exercises in section 2.7 of the supplied PDF.

Lê Sơn Trường — Chương 3, mục 3.1

3.1 — Linear Regression (8 câu)

Câu 1

1. $J(b)=\Sigma(x_i-b)^2$; $J'(b)=2(nb-\Sigma x_i)=0 \Rightarrow b=\text{mean}(x)$;
   $J''=2n>0$ nên cực tiểu duy nhất khi $n>0$.
2. Nếu x_i IID Gaussian cùng mean b, variance cố định, cực đại likelihood tương
   đương cực tiểu bình phương sai số; mean mẫu là MLE của b.
3. Với $\Sigma|x_i-b|$, nghiệm là median; n chẵn có thể là cả khoảng giữa hai giá trị giữa.

Câu 2

Đặt $\tilde{x}=[x;1]$, $\tilde{w}=[w;b]$ thì $\tilde{x}^T\tilde{w}=x^Tw+b$. Affine trên x là tuyến tính trên
không gian đặc trưng mở rộng; không có nghĩa nó tuyến tính thuần túy trên x.

Câu 3

Tạo tầng đặc trưng $[1,x_1,\ldots,x_d,x_1^2,x_1x_2,\ldots,x_d^2]$, rồi tầng tuyến tính cho
đầu ra. Đây là mô hình tuyến tính theo tham số nhưng phi tuyến theo x.
Chỉ ghép các Linear không có tương tác/activation thì không tạo được x_ix_j chính xác.

Câu 4

1. X^TX thiếu hạng: nghịch đảo không tồn tại; nghiệm bình phương tối thiểu vẫn có
   nhưng hệ số có thể không duy nhất.
2. Dùng pseudoinverse/SVD, loại cột phụ thuộc hoặc ridge. Nhiễu Gaussian độc lập
   có thể phá phụ thuộc chính xác khi $n\geq d$; nếu $n<d$ thì hạng vẫn$\leq n$ nên không thể đầy hạng cột.
3. Với X cố định và E_{ij} IID mean 0, variance σ^2:
   $E[(X+E)^T(X+E)]=X^TX+n\sigma^2I$. Đó là kỳ vọng, không bảo đảm một mẫu có điều kiện tốt.
4. SGD có thể hội tụ tới một nghiệm tùy khởi tạo, LR và thành phần null space;
   dự đoán trên train có thể duy nhất dù hệ số không duy nhất. Với zero init và điều
   kiện thích hợp, gradient descent tuyến tính có thể chọn nghiệm norm nhỏ nhất.

Câu 5

Mật độ $(1/2)\exp(-|\varepsilon|)$ trong đề thực ra là Laplace hai phía, không phải
exponential một phía. NLL = $n \log 2 + \Sigma|y_i-x_i^Tw-b|$. Intercept-only có
nghiệm median; hồi quy tổng quát không có công thức nghịch đảo đơn giản, có thể
giải bằng linear programming hoặc subgradient SGD. Một minibatch dùng gradient
sign(yhat−y)*x, trung bình theo batch. Gần nghiệm, gradient có thể đổi dấu và
dao động; giảm LR, averaging hoặc Huber loss giúp ổn định.

Câu 6

Hai tầng affine ghép nhau vẫn affine:
$W_2(W_1x+b_1)+b_2=(W_2W_1)x+(W_2b_1+b_2)$. Không tăng khả năng biểu diễn phi tuyến;
nếu tầng giữa hẹp còn hạn chế hạng của phép biến đổi.

Câu 7

1. Gaussian cộng có thể cho giá âm và variance tuyệt đối cố định, không phù hợp
   khi sai số thường mang tính tỷ lệ.
2. Học log(price) với giá dương giúp mô hình hóa biến động tương đối; exp đầu ra
   cho giá dương. Exp của dự đoán mean log không mặc nhiên là mean giá gốc
   (cần xét bias khi back-transform).
3. Giá giao dịch có bước giá rời rạc; cùng một tick là tỷ lệ thay đổi lớn hơn
   ở giá rất nhỏ. Đây là phân tích mô hình toán học theo đề, không phải tư vấn đầu tư.

Lê Sơn Trường — Chương 3, mục 3.1

Câu 8

1. Số táo là số nguyên không âm; Gaussian liên tục có thể cho số âm/phân số.
2. Với $K \sim \text{Poisson}(\lambda)$: $E[K] = \Sigma k \cdot \lambda^k e^{-\lambda}/k! = \lambda e^{-\lambda} \Sigma[j \geq 0] \lambda^j/j! = \lambda$.
3. NLL mỗi mẫu là $\lambda - k \log \lambda + \log(k!)$, $\lambda > 0$; phần factorial không phụ thuộc tham số.
4. Đặt $\eta = \log \lambda$ thì loss $= \exp \eta - k\eta + \log(k!)$, gradient theo $\eta = \exp \eta - k$.

Ghi nhận chạy thật — mục 3.1

Squared-loss minimizer mean=26.5, absolute-loss minimizers interval [2,3].

Kết quả mục 3.1 — analytic_regression

parameter	estimate
w1	2.00012
w2	-3.40001
bias	4.20021

Lê Sơn Trường — Chương 3, mục 3.2

3.2 — Object-Oriented Design (2 câu)

Câu 1

Mã đầy đủ của HyperParameters, Module, DataModule, Trainer có ở https://d2l.ai/_modules/d2l/torch.html và nguồn d2l/torch.py của D2L 1.0.3. Bài làm lưu bản nguồn tham khảo tại source_reference/d2l_torch.py. HyperParameters giữ cấu hình; Module mô tả mô hình/loss/optimizer; DataModule cung cấp loader; Trainer điều phối forward/backward/update/validation. Các phương thức được thêm dần bằng add_to_class, nên bản trong một đoạn sách có thể chưa đầy đủ như thư viện.

Câu 2

Nếu B.__init__ chỉ nhận a,b mà không gán self.a, self.b, bỏ save_hyperparameters() thì không còn hai thuộc tính đó, truy cập sẽ AttributeError. Hàm tiện ích đọc biến cục bộ của frame gọi rồi gán lên self; tham số hàm không tự trở thành thuộc tính đối tượng. Nếu superclass hoặc code khác đã gán chúng thì cần xét riêng; ví dụ tối thiểu trong notebook không có các phép gán ấy.

Ghi nhận chạy thật — mục 3.2

Without assigning/saving hyperparameters: hasattr(a)=False; hasattr(b)=False

Full reference classes: https://d2l.ai/_modules/d2l/torch.html#HyperParameters; Module; DataModule; Trainer. See downloaded source_reference/d2l_torch.py.

Lê Sơn Trường — Chương 3, mục 3.3

3.3 — Synthetic Regression Data (4 câu)

Câu 1

Nếu N không chia hết B, DataLoader mặc định giữ minibatch cuối nhỏ hơn.
Ví dụ N=101,B=32 → 32,32,32,5; drop_last=True → ba batch 32, bỏ 5 mẫu.
Khi tính loss trung bình cả epoch, phải cân theo số mẫu chứ không trung bình đều batch.

Câu 2

Không đủ RAM: dùng streaming/IterableDataset, memory map hoặc shard trên đĩa.
Shuffle hiệu quả: shuffle thứ tự shard/block, đọc tuần tự từng block và shuffle
trong buffer RAM; hoặc dùng phép hoán vị giả ngẫu nhiên trên chỉ số và gom các
truy cập theo block để giảm seek. Block shuffle chỉ gần tương đương shuffle toàn bộ;
nếu cần hoán vị toàn cục chính xác thì dùng external shuffle nhiều lượt hoặc
khóa ngẫu nhiên + external sort, tốn I/O nhưng không cần giữ toàn bộ bảng hoán vị.

Câu 3

Notebook định nghĩa IterableDataset Fresh: mỗi lần __iter__ sinh x mới bằng
torch.randn, rồi $y=x^{\top}w+b$. Nhãn và đặc trưng được sinh theo từng mẫu, không lưu toàn bộ.

Câu 4

Trong __iter__, tạo generator cục bộ cùng seed cố định rồi truyền vào mọi lần
sinh số. Hai iterator sẽ tạo lại cùng dữ liệu. Khi chạy nhiều worker cần thiết kế
seed/sharding để không vô tình lặp mẫu giữa các worker.

Ghi nhận chạy thật — mục 3.3

Fresh equal on two iterations: False; Fixed equal: True

```
    class Fresh(IterableDataset):
        def __iter__(self):
            for _ in range(3):
                x=torch.randn(2);yield x,2*x[0]-3.4*x[1]+4.2

    class Fixed(IterableDataset):
        def __iter__(self):
            g=torch.Generator().manual_seed(42)
            for _ in range(3):
                x=torch.randn(2,generator=g);yield x,2*x[0]-3.4*x[1]+4.2
```

Kết quả mục 3.3 — last_batch

drop_last	batch_sizes
False	[32, 32, 32, 5]
True	[32, 32, 32]

Lê Sơn Trường — Chương 3, mục 3.4

3.4 — Linear Regression from Scratch (9 câu)

Câu 1

Hồi quy tuyến tính không có các neuron ẩn đồng vai trò nên zero init vẫn học được. Variance khởi tạo 1000 nghĩa là std=√1000, không phải std=1000: dự đoán/loss đầu lớn hơn, có thể mất nhiều bước hoặc mất ổn định với LR lớn. Notebook so sánh zero, std=0.01 (variance=0.0001, cách sinh trong code sách), std=0.1 (variance=0.01 đúng theo chữ "variance" của câu hỏi), và std=√1000 trên cùng dữ liệu. Không được coi variance và standard deviation là một đại lượng.

Câu 2

Định luật Ohm V=RI. Tạo tham số R requires_grad; tối thiểu mean(Vhat−V)² bằng autograd và SGD. Notebook khôi phục R=7 từ dữ liệu giả lập; kết quả đo được in ra, không phải thí nghiệm điện vật lý.

Câu 3

Giữ λ và h,c,k đúng đơn vị SI; tham số hóa T=exp θ để T>0. Fit phổ giả lập Planck ở nhiều bước sóng bằng loss trên log B; tính expm1 để ổn định khi số mũ nhỏ. Notebook học T từ khởi tạo 3000 K đến gần nhiệt độ thật 4500 K.
Với dữ liệu đo thực, cần thêm nhiễu, hiệu chuẩn và có thể hệ số phát xạ; không khẳng định một phép fit đơn giản xác định mọi vật thể.

Câu 4

Backward mặc định không giữ graph của gradient. Dùng autograd.grad(..., create_graph=True) để đạo hàm tiếp. Hessian n×n tốn bộ nhớ và thời gian; HVP, autograd.functional.hessian, hoặc xấp xỉ bậc hai phù hợp hơn tùy mục tiêu.
ReLU tại điểm gãy không có đạo hàm cổ điển; không coi mọi Hessian là tồn tại trơn.

Câu 5

yhat shape (B,1), y shape (B,) nếu trừ trực tiếp sẽ broadcast thành (B,B). y.reshape(yhat.shape) bảo đảm mỗi dự đoán so đúng một nhãn.

Câu 6

Notebook vẽ loss với LR 0.003, 0.03, 0.3 trong 10 epoch. LR nhỏ học chậm; LR lớn có thể nhanh nhưng vượt ngưỡng ổn định thì dao động/phân kỳ.
Thêm epoch có thể giảm lỗi tối ưu, không bảo đảm giảm lỗi validation mãi.
Các giá trị này là khảo sát theo yêu cầu, không đổi baseline 3 epoch, LR=0.03 của sách.

Câu 7

Iterator giữ batch cuối nhỏ hơn nếu không drop_last. Nếu loss dùng tổng và chia cho batch_size cố định, batch cuối bị scale sai; dùng trung bình theo số mẫu thực.

Câu 8

Notebook chạy squared/L1/Huber trên cùng dữ liệu sạch và dữ liệu có y[5]=10000, so sánh sai số trọng số và bias, không so trực tiếp trị số giữa các loss khác đơn vị.
Squared loss có gradient theo residual nên outlier chi phối; L1 có gradient biên độ không đổi nhưng khó tối ưu gần 0. Huber bậc hai gần 0, tuyến tính khi residual lớn, giúp kết hợp tính trơn gần nghiệm và độ bền trước outlier.

Lê Sơn Trường — Chương 3, mục 3.4

Câu 9

Shuffle làm minibatch đại diện hơn, giảm thiên lệch do thứ tự. Ví dụ x=1,
nhãn gồm một block +1 rồi một block −1; SGD LR lớn có xu hướng chạy theo block
cuối ở mỗi epoch. Không phải mọi dữ liệu không shuffle đều thất bại, nhưng thứ tự
đối nghịch có thể gây dao động hoặc hội tụ chậm; shuffle không thay thế việc chọn LR hợp lý.

Ghi nhận chạy thật — mục 3.4

Ohm fitted R=7.000000, true R=7

Planck fitted T=4500.000 K, true 4500 K; synthetic noiseless spectrum

Initialization: std=0.01 means variance=0.0001. Literal variance=0.01 in the exercise is tested separately with std=0.1; variance=1000 uses std=sqrt(1000).

Kết quả mục 3.4 — initialization

init_std	init_variance	loss	w_error
0	0	0.0600349	0.263653
0.01	0.0001	0.0599398	0.263324
0.1	0.01	0.0591148	0.260457
31.6228	1000	3.37111	2.53765

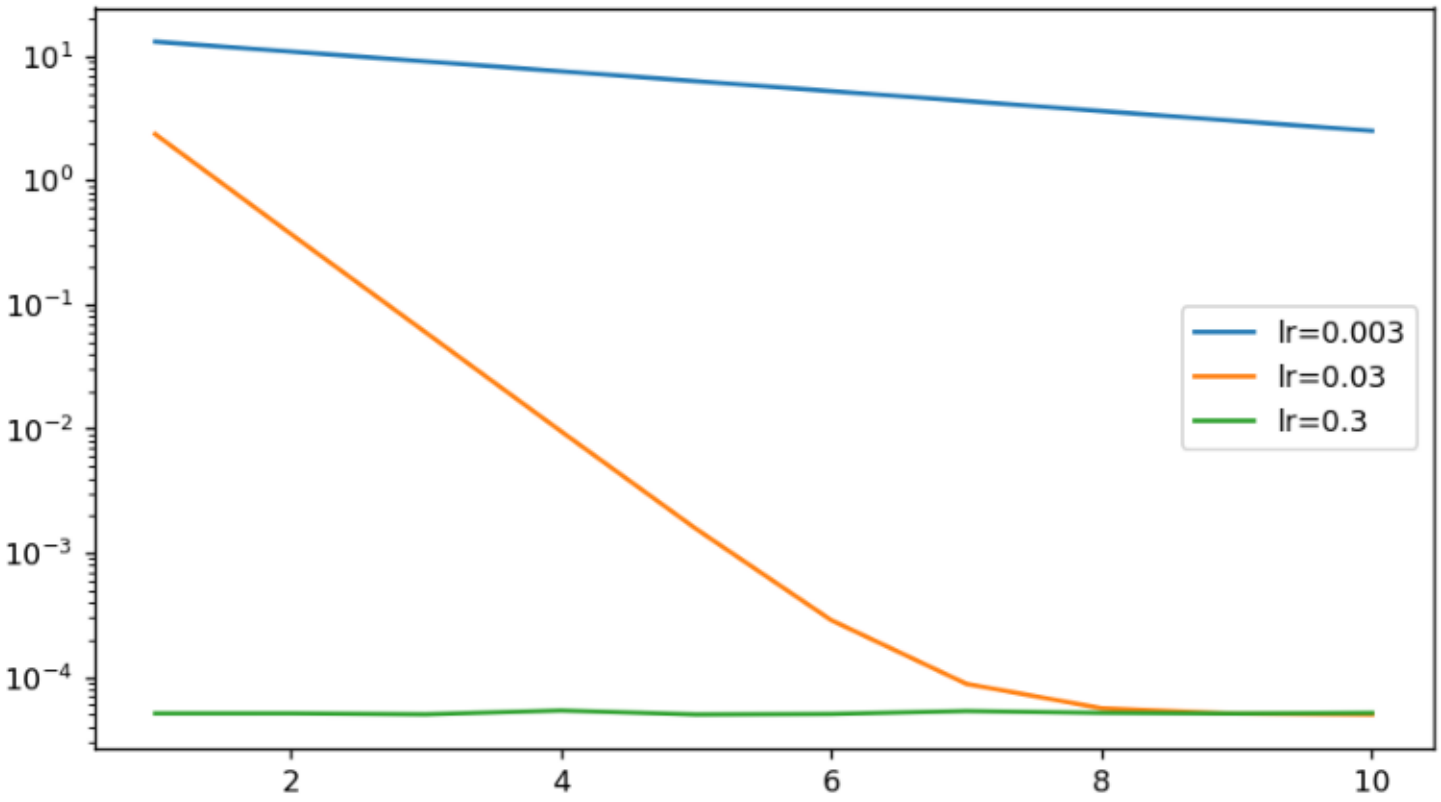
Kết quả mục 3.4 — learning_rates

lr	final_loss
0.003	2.51533
0.03	5.01048e-05
0.3	5.16489e-05

Kết quả mục 3.4 — robust_losses

outlier	loss	w_error	bias_error
False	mse	0.00039417	0.000239563
False	l1	0.0176247	0.00187511
False	huber	0.0388477	0.0230343
True	mse	13.9257	10.1756
True	l1	0.0216266	0.00749807
True	huber	0.038209	0.0222122

learning_rates



Lê Sơn Trường — Chương 3, mục 3.5

3.5 — Concise Linear Regression (5 câu)

Câu 1

Nếu cập nhật cũ dùng gradient tổng trực tiếp với LR η, đổi loss thành mean
thì gradient giảm B lần; muốn cùng bước phải đặt η_mean=Bη_sum.
Nếu code cũ đã chia gradient tổng cho B trong optimizer thì mean loss dùng cùng η,
không chia thêm lần nữa. Phải kiểm tra reduction và bước update cùng nhau.

Câu 2

Công thức của sách là scaled Huber: $e^2/(2\sigma)$ nếu $|e|\le\sigma$, và $|e|-\sigma/2$ nếu ngoài.
torch.nn.HuberLoss(delta=σ) cho $e^2/2$ và $\sigma(|e|-\sigma/2)$, nên chia cho σ
để khớp sách; SmoothL1Loss(beta=σ) cũng khớp. Với σ=1 chúng trùng nhau.
Notebook dùng delta=1 và so sánh trong mục 3.4, đồng thời chạy bản framework ở mục này.

Câu 3

Sau backward, đọc model.weight.grad hoặc model.net[...].weight.grad.
Gradient chưa có trước backward, bị reset khi zero_grad, và cộng dồn nếu không reset.

Câu 4

LR/epoch quyết định mức hội tụ và ổn định. Thêm epoch không cải thiện vô hạn:
có đáy nhiễu, độ chính xác số, và có thể overfitting. Xem đường học của 3.4;
không dùng training loss làm bằng chứng duy nhất về tổng quát hóa.

Câu 5

Notebook vẽ $\|\hat{w}-w\|_2$ và $|\hat{b}-b|$ theo n=5,10,20,50,...,10000, lấy trung bình
10 seed dữ liệu. Đường chính chạy mô hình framework với SGD LR=0.03, 3 epoch,
batch32 như ví dụ; có thêm least-squares chính xác làm tham chiếu để tách
tác động thống kê khỏi lỗi chưa hội tụ. Cùng số epoch nhưng n lớn hơn cũng có
nhiều bước SGD hơn. Với mô hình đúng và đủ điều kiện, lỗi thống kê thường giảm cỡ $1/\sqrt{n}$,
nhưng từng mẫu có thể không giảm đều. Dải n theo log bao phủ nhiều bậc độ lớn,
giúp quan sát hiệu quả tăng dữ liệu thay vì chỉ một vùng hẹp.

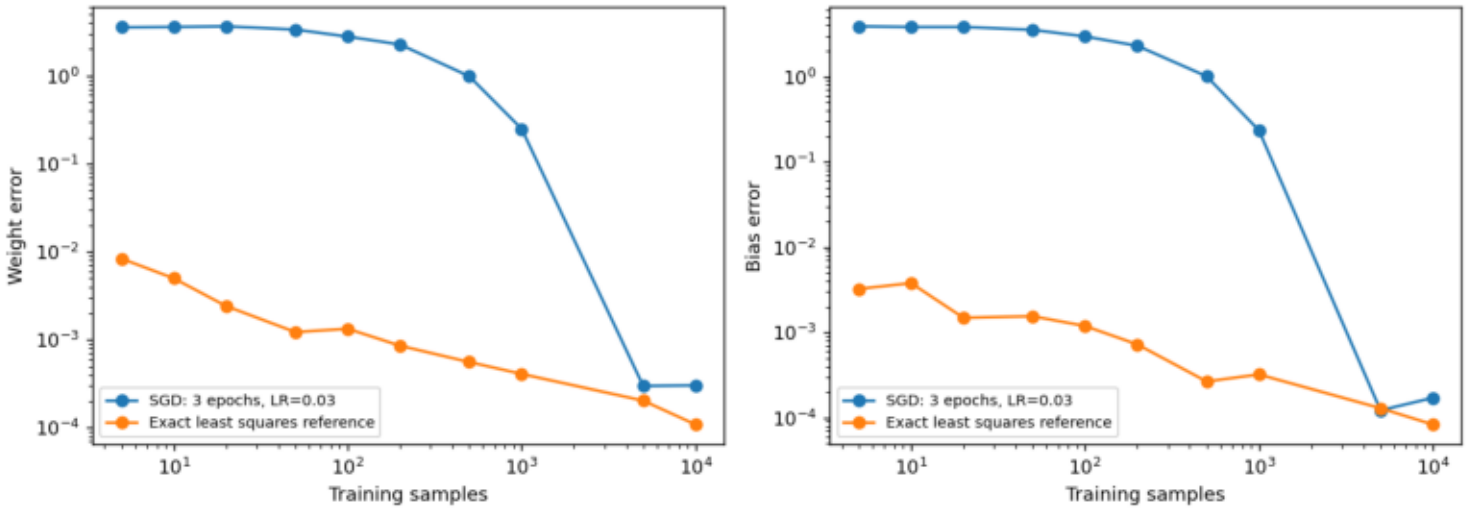
Ghi nhận chạy thật — mục 3.5

Framework Huber final loss=2.2865465; weight.grad=[[-0.20225602388381958, 0.09151758253574371]]

Sample-size plot includes framework SGD with the book's LR=0.03 and 3 epochs, plus exact least squares as a separate reference. Ten data seeds per size. Larger n also means more SGD updates at fixed epochs.

Kết quả mục 3.5 — data_size

n	sgd_w_error_mean	sgd_bias_error_mean	ols_w_error_mean	ols_bias_error_mean
5	3.53928	3.88286	0.00835432	0.00323682
10	3.57702	3.81553	0.00499011	0.00378426
20	3.63359	3.8167	0.00241498	0.0014842
50	3.33586	3.52884	0.00122347	0.00154706
100	2.77474	2.95418	0.00133428	0.00118961
200	2.25693	2.29142	0.00085607	0.000722656
500	0.98307	1.0072	0.000558388	0.000265989
1000	0.248826	0.233757	0.000409327	0.00032135
5000	0.000299042	0.000121984	0.00020356	0.000129156
10000	0.00030343	0.000170937	0.000108973	8.30746e-05


Weight error
Bias error
Training samples
Training samples
SGD: 3 epochs, LR=0.03
Exact least squares reference
SGD: 3 epochs, LR=0.03
Exact least squares reference

Lê Sơn Trường — Chương 3, mục 3.6

3.6 — Generalization (7 câu)

Câu 1

Hồi quy đa thức bậc d giải bình phương tối thiểu bằng ma trận Vandermonde. Nếu n=d+1 điểm x phân biệt thì có đa thức nội suy duy nhất; n≤d+1 có thể nội suy nhưng chưa chắc duy nhất. Khi n>d+1 chỉ khớp chính xác nếu nhãn nằm trong không gian cột; thiếu hạng dùng pseudoinverse. Khớp train chính xác không bảo đảm tổng quát hóa.

Câu 2

Năm ví dụ không nên chia ngẫu nhiên như IID: chuỗi thời gian liền kề; ảnh nhiều khung của cùng video; nhiều lần đo một bệnh nhân; văn bản của cùng tác giả/tài liệu; đo đạc địa lý gần nhau. Dùng time split, group split hoặc spatial split tương ứng.

Câu 3

Có thể zero train error khi mô hình nội suy/ghi nhớ, hoặc dữ liệu không nhiễu và mô hình đúng. Zero generalization error đòi hỏi nhiệm vụ xác định và hàm học đúng trên phân phối đánh giá; nhiễu không loại bỏ tạo ra Bayes error dương. Với classification, zero 0–1 error khác với cross-entropy đúng bằng 0.

Câu 4

K-fold huấn luyện K mô hình cho mỗi cấu hình; H cấu hình cần khoảng K·H lần fit. Tiền xử lý cũng phải fit riêng từng fold. Đó là chi phí lớn khi mạng sâu học lâu.

Câu 5

CV đánh giá mô hình học trên (K−1)/K dữ liệu, không phải mô hình cuối học toàn bộ; cỡ train nhỏ hơn thường gây thiên lệch bi quan, nhưng dấu bias không tuyệt đối trong mọi thuật toán. Thử nhiều cấu hình và chọn min CV còn có selection bias lạc quan.

Câu 6

VC dimension đếm khả năng shatter nhãn, bỏ qua norm, margin, độ trơn và thuật toán chọn nghiệm. Hai lớp cùng VC dimension có thể có mức tổng quát hóa thực tế khác; với hồi quy thực liên tục còn cần các khái niệm như pseudo-dimension/Rademacher complexity.

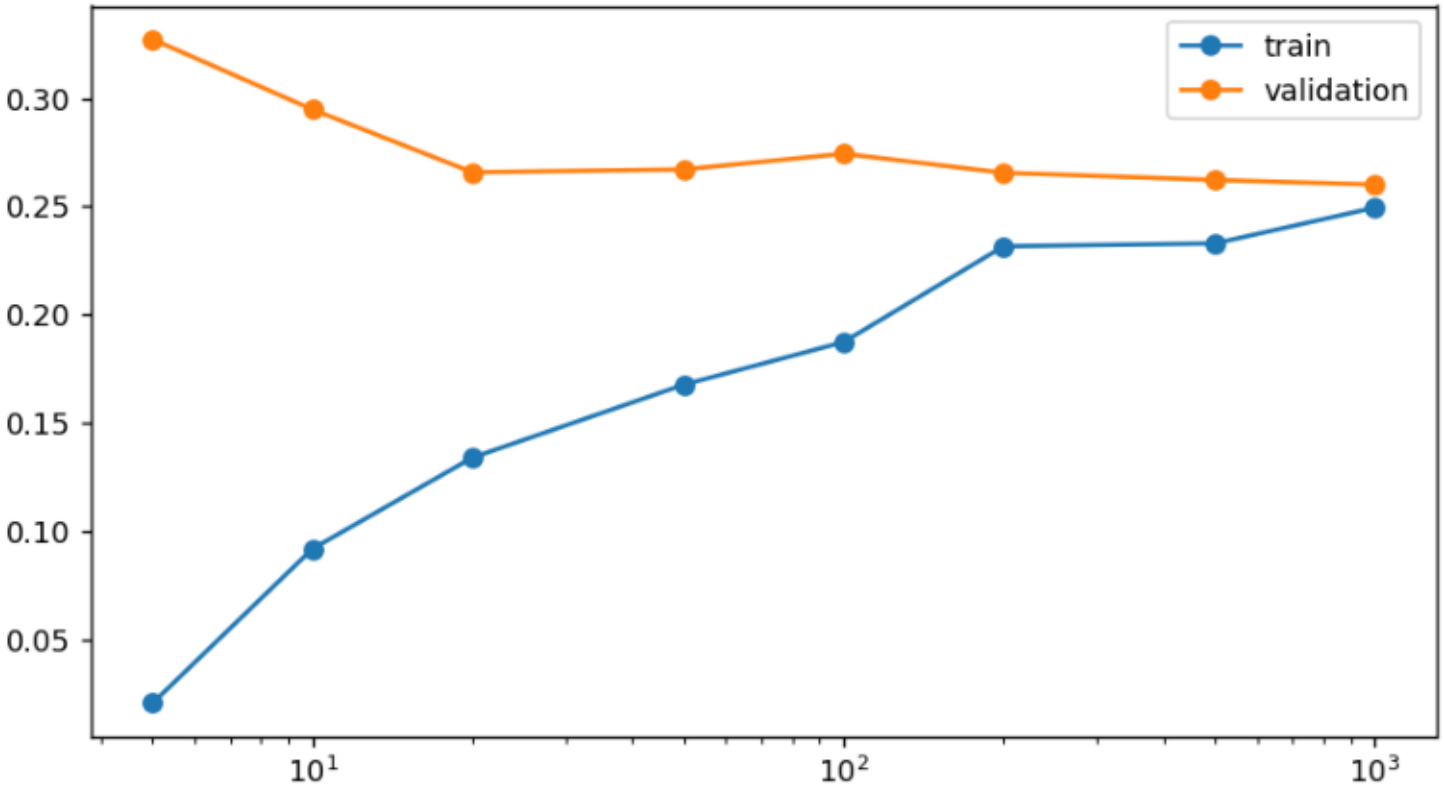
Câu 7

Giảm tập train thành các kích thước khác nhau, giữ validation cố định và vẽ learning curve. Nếu val error còn giảm theo n trong khi train/val có khoảng cách, đó là bằng chứng thêm dữ liệu có thể giúp. Nếu cả hai lỗi cao và phẳng, cần cải thiện mô hình/đặc trưng/tối ưu, chưa thể chỉ yêu cầu thêm dữ liệu. Notebook thực hiện đường học này.

Kết quả mục 3.6 — learning_curve

n	train_mse	validation_mse
5	0.0207186	0.327282
10	0.0919908	0.294855
20	0.134089	0.26588
50	0.167793	0.267184
100	0.18746	0.274405
200	0.231668	0.265642
500	0.233009	0.262312
1000	0.249522	0.260194

Lê Sơn Trường — Kết quả mục 3.6
data_learning_curve



Lê Sơn Trường — Chương 3, mục 3.7

3.7 — Weight Decay (6 câu)

Câu 1

Đề viết “accuracy” trong bài hồi quy; ở đây báo cáo train/validation MSE cho đúng đại lượng. Dữ liệu đúng mục 3.7: $y=0.05+\Sigma 0.01x_i+\varepsilon$, $\text{std}(\varepsilon)=0.01$, 20 train/100 validation, $d=200$. Đường chính dùng SGD, batch5, LR=0.01, 10 epoch, khảo sát $\lambda=0/0.01/0.1/1/3/10$; không phạt bias.
Ridge quá yếu có thể overfit, quá mạnh có thể underfit. Bảng ridge giải chính xác ở lưới 0...100 chỉ là tham chiếu riêng, không thay cho kết quả SGD của mục này.

Câu 2

Chọn λ có validation MSE thấp nhất trong lưới đã thử. Đó không bảo đảm tối ưu trên mọi λ hoặc dữ liệu mới vì validation hữu hạn và đã dùng chọn mô hình.
Giá trị đủ tốt/ổn định thường quan trọng hơn λ chính xác tuyệt đối; cần test độc lập hoặc nested CV nếu cần ước lượng tổng quát hóa sau chọn tham số.

Câu 3

Nếu giữ hệ số $\lambda/2$ như biểu thức sách và thay bình phương norm bằng $\Sigma|w_i|$, $J=L+(\lambda/2)\Sigma|w_i|$: cập nhật $w\leftarrow w-\eta(\nabla L+(\lambda/2)\text{sign}(w))$, chọn subgradient trong $[-1,1]$ tại 0. Proximal gradient cho nghiệm thưa rõ hơn: $w\leftarrow \text{sign}(z)\ \max(|z|-\eta\lambda/2,0)$, $z=w-\eta\nabla L$. Nếu định nghĩa lại $J=L+\lambda\Sigma|w_i|$, hệ số trong gradient và ngưỡng là λ thay vì $\lambda/2$. Bias có thể không phạt.

Câu 4

$\|W\|^2_F=\Sigma_{ij}W_{ij}^2=\text{tr}(W^TW)=\text{tr}(WW^T)$, tương tự w^Tw cho vector.

Câu 5

Có thể thêm dữ liệu phù hợp, augmentation bảo toàn nhãn, early stopping, dropout, giảm leakage và chọn đặc trưng. Tăng thời gian train tự nó không phải thuốc chống overfit; phải theo dõi validation. Những biện pháp có thể không cộng hiệu quả tuyến tính.

Câu 6

MAP cực đại log likelihood + log prior, tương đương NLL − log prior.
Prior Gaussian mean 0 tạo L2; Laplace tạo L1. Hệ số λ phụ thuộc variance prior, quy ước tổng/trung bình loss và cỡ mẫu; không đồng nhất mọi optimizer weight decay với cùng một MAP (AdamW tách decay là trường hợp cần phân biệt).

Ghi nhận chạy thật — mục 3.7

Regression has no classification accuracy: report unpenalized MSE. Main curve is SGD, LR=.01, batch=5, 10 epochs, true bias=.05, no bias penalty. Exact ridge is a separate reference, not the SGD result.

Kết luận thực nghiệm — mục 3.7

Kết quả SGD đúng dữ liệu mục 3.7: bias thật 0.05, 20 train/100 validation, batch5, LR=0.01, 10 epoch. λ=3 có validation MSE thấp nhất lưới (0.021553); λ=0 cho 0.032820. Bảng exact ridge là tham chiếu riêng.

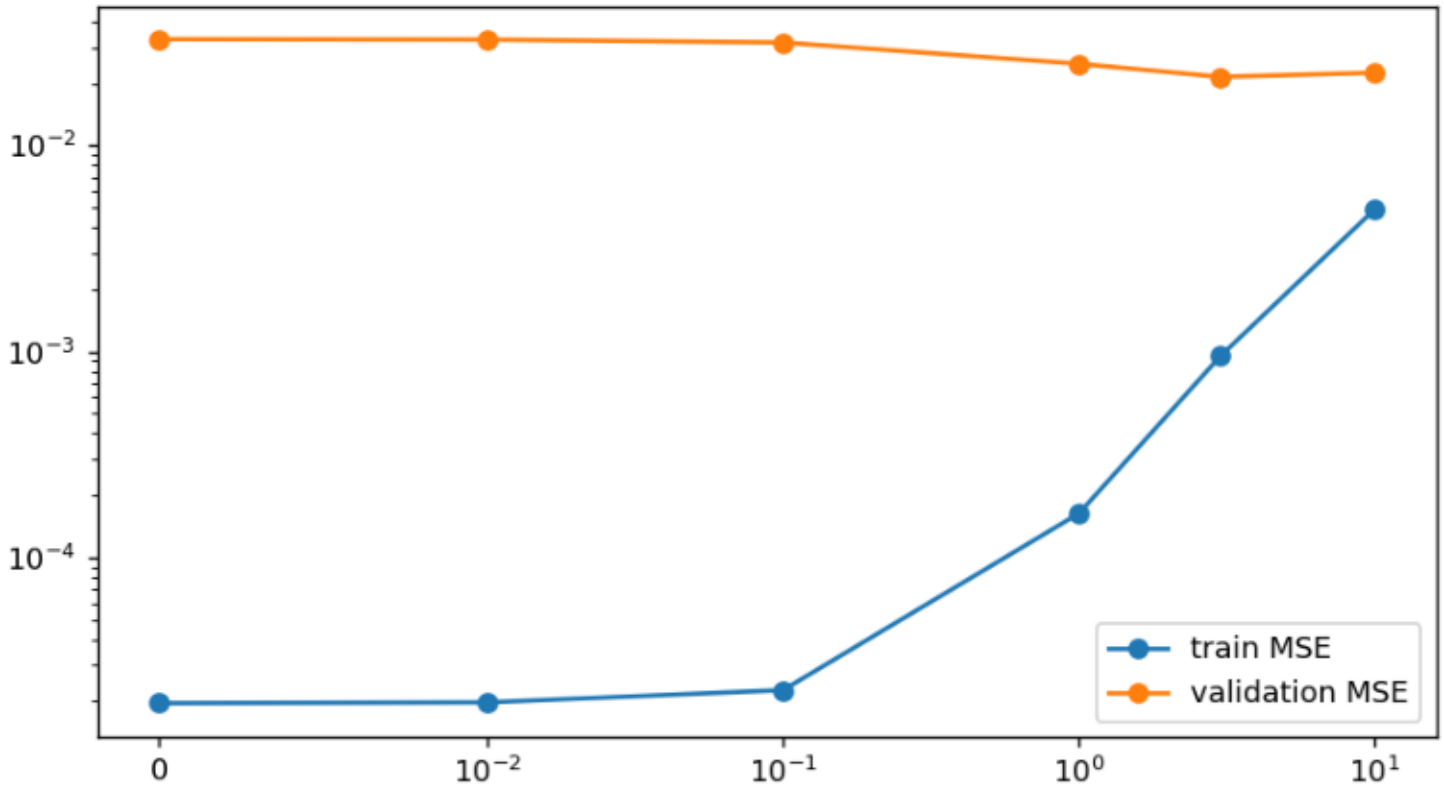
Kết quả mục 3.7 — weight_decay

lambda	train_mse	validation_mse	weight_norm
0	1.95968e-05	0.0328204	0.130827
0.01	1.97738e-05	0.0326978	0.130355
0.1	2.26808e-05	0.031645	0.126192
1	0.000163772	0.0249405	0.0921735
3	0.00094848	0.0215526	0.051297
10	0.00492136	0.0225684	0.0242086

Kết quả mục 3.7 — exact_ridge_reference

lambda	train_mse	validation_mse	weight_norm
0	2.87662e-32	0.0237244	0.0459817
1e-05	2.14125e-14	0.0206284	0.0451934
0.0001	2.14121e-12	0.0206283	0.0451929
0.001	2.14077e-10	0.0206281	0.0451887
0.01	2.13639e-08	0.0206254	0.0451461
0.1	2.09341e-06	0.0205995	0.0447251
1	0.000173189	0.0203872	0.0409486
10	0.00504169	0.0198603	0.0225812
100	0.0168779	0.020158	0.00418584

regularization_curve



Lê Sơn Trường — Chương 4, mục 4.1

4.1 — Softmax Regression (7 câu)

Câu 1

Với one-hot y, $L=\log\Sigma\exp(o_i)-y^To$, $p=\text{softmax}(o)$: $\nabla L=p-y$.
Hessian $H=\text{diag}(p)-pp^T$; $H_{ij}=p_i(\delta_{ij}-p_j)$.
Nếu Z là vector one-hot ngẫu nhiên theo p thì $E[ZZ^T]=\text{diag}(p)$, nên
$\text{Cov}(Z)=\text{diag}(p)-pp^T=H$. Với vector v, $v^THv=\text{Var}(v^TZ)\geq 0$;
notebook kiểm chứng Hessian autograd và covariance trùng nhau.

Câu 2

Ba lớp đều nhau có entropy $\log_2 3\approx 1{,}585$ bit/ký hiệu, không gán được số bit nguyên
cố định bằng entropy. Mã nhị phân cố định cần 2 bit, lãng phí một mã; mã prefix
0,10,11 có kỳ vọng 5/3 bit. Ghép hai quan sát có 9 khả năng: Huffman dùng bảy mã
dài 3 bit và hai mã dài 4 bit (000,...,110,1110,1111), trung bình 29/9 bit/cặp,
tức $29/18\approx 1{,}611$ bit/ký hiệu, tốt hơn 5/3. Nếu bắt buộc mã độ dài cố định,
cần $\text{ceil}(\log_2 9)=4$ bit/cặp, chưa tốt hơn fixed một ký hiệu; cần phân biệt hai loại mã.
Ghép n quan sát cần $\text{ceil}(n\log_2 3)$ bit, trung bình $\to \log_2 3$; ví dụ n=5 cần 8 bit,
tức 1,6 bit/ký hiệu. Có thể dùng arithmetic coding để khai thác xác suất.

Câu 3

Hai trit đủ vì $3^2=9\geq 8$; một trit chỉ có 3 trạng thái. Ba mức điện áp mang
$\log_2 3$ bit mỗi symbol, có thể giảm symbol rate hoặc số đường truyền cho cùng dữ liệu.
Đổi lại phân biệt nhiều mức làm biên nhiễu nhỏ hơn; không khẳng định luôn tốt hơn nhị phân.

Câu 4

Softmax $p_i=e^{o_i}/\Sigma e^{o_j}$ tăng theo o_i khi giữ các điểm khác cố định:
$\partial p_i/\partial o_i=p_i(1-p_i)>0$ nếu có hơn một lựa chọn xác suất dương.
$o_i>o_j \Leftrightarrow p_i>p_j$. Thêm lựa chọn "không chọn" như lớp thứ ba với score riêng
(có thể đặt 0 làm mốc); chuẩn hóa lại trên cả ba lớp, thay vì ép hai lựa chọn có tổng 1.

Câu 5

Đặt $m=\max(a,b)$, $d=|a-b|$. $\text{LSE}=m+\log(1+e^{-d})>m$ với a,b hữu hạn.
Chênh lệch trong $(0,\log 2]$, bằng $\log 2$ khi $a=b$ và tiến về 0 khi $d\to\infty$;
0 là infimum, không đạt với hai số hữu hạn.
Với $\lambda>0$: $\text{LSE}(\lambda a,\lambda b)/\lambda=m+\log(1+e^{-\lambda^d})/\lambda$;
sai số$\leq \log 2/\lambda\to 0$. Softmin $=-\log(e^{-a}+e^{-b})$, hoặc phiên bản nhiệt độ
$-\log\Sigma\exp(-\lambda x_i)/\lambda$. Với k số, sai số softmax$\leq \log k/\lambda$.

Câu 6

1. Gradient log-partition là softmax, Hessian là covariance PSD ở câu 1, nên g convex.
2. Phát biểu đề có lỗi: $g(x+b\cdot 1)=b+g(x)$, không phải $g(x)$.
   Softmax$(x+b\cdot 1)=$softmax(x) mới là bất biến. Notebook đưa phản ví dụ b=3.
3. Logits dương lớn có thể overflow exp; cùng âm lớn có thể underflow về 0,
   dẫn tới chia 0/log 0. Chỉ trừ max trước exp không thay đổi softmax.
4. Đặt $m=\max x$: $g(x)=m+\log\Sigma\exp(x_i-m)$; số mũ$\leq 0$ và có ít nhất một số bằng 1.
   Khi tính cross-entropy dùng log_softmax/logsumexp, tránh tính p rồi log p quá nhỏ.

Câu 7

$Q_i\propto P_i\text{^}\alpha$ tương ứng temperature $T=1/\alpha$ so với T=1. Gấp đôi nhiệt độ $\to \alpha=1/2$;
giảm một nửa $\to \alpha=2$. $T\to 0$ tập trung đều trên những lớp đồng cực đại (nếu chỉ

một cực đại thì one-hot). $T \to \infty$ đều trên support $P_i > 0$; phần tử $P_i = 0$ luôn bằng 0 với mọi $\alpha > 0$. Với logits hữu hạn thì mọi xác suất dương nên giới hạn đều trên mọi lớp.

Ghi nhận chạy thật — mục 4.1

logsumexp(x+3)-logsumexp(x)=3.0; softmax invariant=True

Kết quả mục 4.1 — hessian

```
          0         1         2
  0.164691 -0.0583573 -0.106334
-0.0583573   0.201893 -0.143536
 -0.106334  -0.143536   0.24987
```

Kết quả mục 4.1 — one_hot_covariance

```
          0         1         2
  0.164691 -0.0583573 -0.106334
-0.0583573   0.201893 -0.143536
 -0.106334  -0.143536   0.24987
```

Lê Sơn Trường — Chương 4, mục 4.2

4.2 — Image Classification Dataset (3 câu)

Câu 1

Notebook đọc đủ 60.000 mẫu với batch 1/32/256/1024, đo samples/s.
Batch=1 thường có overhead Python/collate cao trên mỗi mẫu; kết luận định lượng
lấy từ bảng thực nghiệm, không chỉ suy đoán. Dữ liệu đã giải nén trong RAM, nên
phép đo này không đại diện tốc độ tải Internet hay đọc ảnh JPEG từ đĩa.

Câu 2

Notebook dùng cProfile để tìm các hàm tốn thời gian, lưu top cumulative time.
Có thể batch fetch/collate, cache biến đổi, dùng worker hoặc pin_memory khi chuyển GPU.
Với tensor đã nằm RAM, worker/process overhead có thể lớn hơn lợi ích; cần đo
thay vì mặc định càng nhiều worker càng nhanh. Trên Windows phải đặt entry point
trong if __name__=='__main__' khi tạo worker ngoài notebook.

Câu 3

API torchvision có MNIST, FashionMNIST, CIFAR10, CIFAR100, SVHN, ImageFolder,
ImageNet... Theo tài liệu torchvision (https://docs.pytorch.org/vision/stable/datasets.html).
Trong bài làm đọc trực tiếp file IDX chính thức để không phụ thuộc cặp phiên bản
torch/torchvision; dữ liệu vẫn là Fashion-MNIST gốc, không thay bằng digits hay make_moons.

Ghi nhận chạy thật — mục 4.2

```
Fashion-MNIST shapes: train=(60000, 784), labels=(60000,), test/validation=(10000, 784)

         374200 function calls (373728 primitive calls) in 0.219 seconds

   Ordered by: cumulative time
   List reduced from 66 to 12 due to restriction <12>

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      236    0.000    0.000    0.218    0.001
C:\Users\LeSonTruong\Documents\GitHub\DeepL\Buoi1\Buoi1_PyTorch\.venv\Lib\site-
packages\torch\utils\data\dataloader.py:720(__next__)
      236    0.011    0.000    0.215    0.001
C:\Users\LeSonTruong\Documents\GitHub\DeepL\Buoi1\Buoi1_PyTorch\.venv\Lib\site-
packages\torch\utils\data\dataloader.py:783(_next_data)
      235    0.007    0.000    0.203    0.001
C:\Users\LeSonTruong\Documents\GitHub\DeepL\Buoi1\Buoi1_PyTorch\.venv\Lib\site-
packages\torch\utils\data\_utils\fetch.py:49(fetch)
    60000    0.037    0.000    0.143    0.000
C:\Users\LeSonTruong\Documents\GitHub\DeepL\Buoi1\Buoi1_PyTorch\.venv\Lib\site-
packages\torch\utils\data\dataset.py:205(__getitem__)
   180000    0.106    0.000    0.106    0.000
C:\Users\LeSonTruong\Documents\GitHub\DeepL\Buoi1\Buoi1_PyTorch\.venv\Lib\site-
packages\torch\utils\data\dataset.py:206(<genexpr>)
      235    0.000    0.000    0.053    0.000
C:\Users\LeSonTruong\Documents\GitHub\DeepL\Buoi1\Buoi1_PyTorch\.venv\Lib\site-
packages\torch\utils\data\_utils\collate.py:340(default_collate)
   705/235    0.003    0.000    0.053    0.000
C:\Users\LeSonTruong\Documents\GitHub\DeepL\Buoi1\Buoi1_PyTorch\.venv\Lib\site-
packages\torch\utils\data\_utils\collate.py:118(collate)
      470    0.001    0.000    0.038    0.000
C:\Users\LeSonTruong\Documents\GitHub\DeepL\Buoi1\Buoi1_PyTorch\.venv\Lib\site-
packages\torch\utils\data\_utils\collate.py:246(collate_tensor_fn)
      470    0.037    0.000    0.037    0.000 {built-in method torch.stack}
      235    0.003    0.000    0.012    0.000 {built-in method builtins.all}
    60000    0.006    0.000    0.009    0.000
C:\Users\LeSonTruong\Documents\GitHub\DeepL\Buoi1\Buoi1_PyTorch\.venv\Lib\site-
packages\torch\utils\data\_utils\collate.py:207(<genexpr>)
    60001    0.003    0.000    0.003    0.000 {built-in method builtins.len}
```

API datasets include MNIST, FashionMNIST, CIFAR10/100, ImageFolder, SVHN (torchvision docs). Here IDX is read directly to avoid torch/torchvision ABI mismatch; no data substitution.

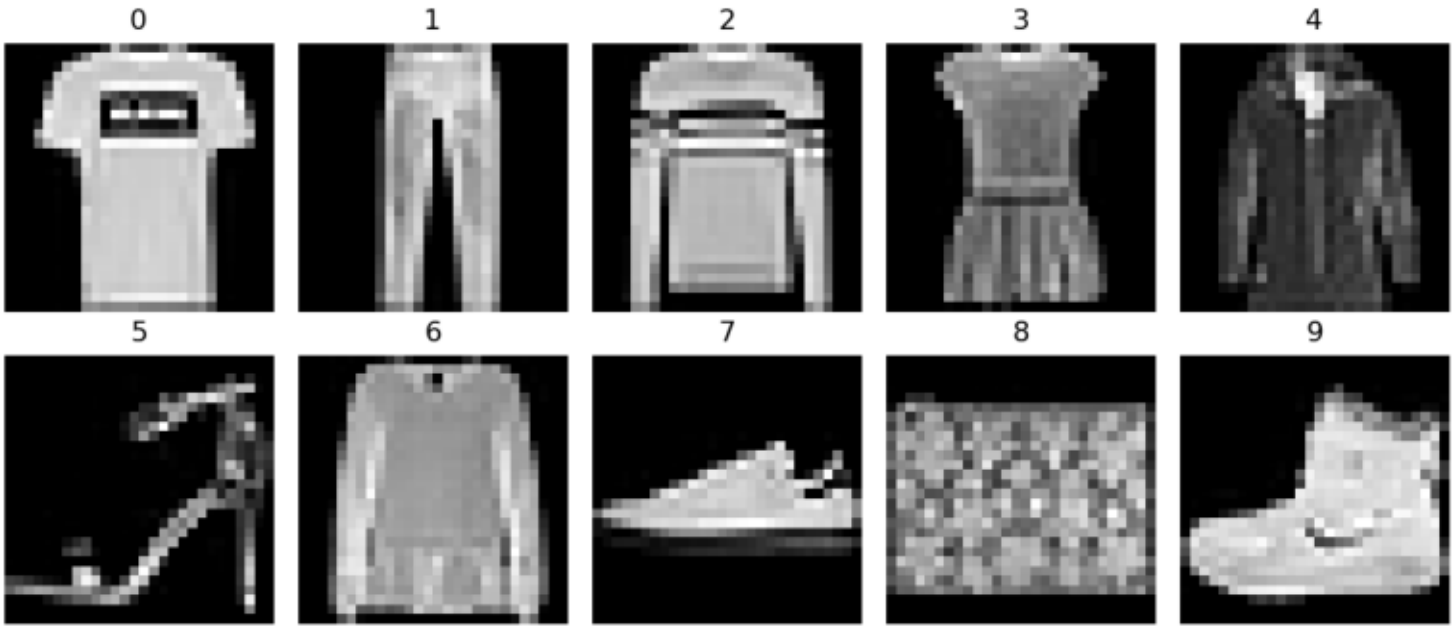
Kết quả mục 4.2 — data_iterator_speed

batch	samples	seconds	samples_per_second
1	60000	0.991204	60532.4
32	60000	0.190848	314386
256	60000	0.168547	355984
1024	60000	0.257092	233379

Kết quả mục 4.2 — batch_fetch_optimization

method	samples	seconds	samples_per_second
tensor split views, already in RAM	60000	0.0004387	1.36768e+08

fashion_examples



Lê Sơn Trường — Chương 4, mục 4.3

4.3 — Base Classification Model (3 câu)

Câu 1

Gọi N số mẫu, B số batch, m batch size chuẩn, b=N−(B−1)m là cỡ batch cuối.
Lq là trung bình không trọng số các batch mean; l_last là mean batch cuối.
Tổng loss các batch đầy là m(B·Lq−l_last); tổng cuối b·l_last.
Do đó Lv=[mB·Lq−(m−b)l_last]/N. Khi b=m thì Lv=Lq.
Notebook kiểm tra với N=11, m=4, b=3.

Câu 2

Nếu thứ tự mẫu là hoán vị ngẫu nhiên độc lập với loss, mọi batch mean đều có
kỳ vọng bằng mean toàn tập. Vậy E[Lq]=E[Lv]. Nhưng với một thứ tự cố định,
Lq có thể lệch và phương sai lớn hơn vì vài mẫu cuối có trọng số cao hơn.
Lv cân theo số mẫu cho giá trị chính xác của tập đó và không phụ thuộc cách chia batch.
Nếu dữ liệu không shuffle và batch cuối khác phân phối, không mặc định Lq unbiased.

Câu 3

Chọn yhat=argmin[a] Σ_y p(y|x) l(y,a) để tối thiểu expected loss.
Với 0–1 loss là argmax xác suất. Khi chi phí sai khác nhau, quyết định có thể
khác lớp nhiều khả năng nhất; cần loss/cost matrix phù hợp bài toán.

Kết quả mục 4.3 — batch_average

n	batch_size	last_batch_size	quick_mean	weighted_mean	corrected_mean
11	4	3	5.33333	5	5

Lê Sơn Trường — Chương 4, mục 4.4

4.4 — Softmax from Scratch (5 câu)

Câu 1

Notebook chạy logits [100,101,102] và [−101,−102,−103] ở FP32. Exp trực tiếp có thể tràn hoặc underflow, sinh NaN/đáp án sai do subnormal. Sửa bằng trừ max mỗi hàng rồi exp/normalize; để tính loss dùng logsumexp cho ổn định hơn nữa.

Câu 2

Cross-entropy đầy đủ là $-\Sigma_i\, y_i \log p_i$ (cần dấu âm; bản in câu hỏi bỏ dấu âm).
Notebook thực hiện bản one-hot và đối chiếu với fused/index CE trên cùng logits.
Dense one-hot cần thao tác trên mọi lớp, thường tốn bộ nhớ/thao tác hơn gather nhãn nguyên, nhưng tốc độ cụ thể phải đo; bảng có benchmark sau warm-up.
Dense CE hữu ích với soft labels/label smoothing/distillation. Cẩn thận p=0, log 0 và biểu thức $0\cdot(-\infty)$ gây NaN; dùng logits và log_softmax thay vì nhân trực tiếp.

Câu 3

Không phải lúc nào argmax cũng tối ưu: có bài toán chi phí false negative và false positive khác nhau. Dùng expected cost, ngưỡng theo chi phí, hiệu chuẩn xác suất hoặc cơ chế từ chối/chuyển kiểm tra. Đây là trả lời nguyên tắc phân loại, không đưa ra ngưỡng chẩn đoán y khoa cụ thể.

Câu 4

Vocabulary lớn làm logits/normalization và ma trận trọng số đầu ra lớn, lớp hiếm ít dữ liệu, tăng thời gian và RAM. Có thể dùng subword, tying embeddings, sampled/hierarchical/adaptive softmax tùy mục tiêu. Sampling khi train không luôn cho đúng normalization tại inference; phải phân biệt xác suất chuẩn hóa và điểm xếp hạng.

Câu 5

Notebook huấn luyện bản scratch (W,b và phép tính logits/CE rõ ràng) với LR=0.01/0.1/1, batch mặc định 256, 10 epoch; sau đó batch=32/1024 với LR=0.1.
Vẽ validation loss theo epoch và ghi cả train loss/accuracy. Batch nhỏ làm nhiều bước cập nhật hơn mỗi epoch; không coi so sánh cùng epoch là cùng số bước hay thời gian.
Kết luận lấy từ bảng kết quả đo, không đặt trước một LR luôn tốt nhất.

Ghi nhận chạy thật — mục 4.4

Dense one-hot CE=2.820356; fused/index CE=2.820356; negative sign is required.

Kết luận thực nghiệm — mục 4.4

Nhận xét từ số đo: batch_32 có validation loss thấp nhất trong lưới softmax (0.4593), accuracy 84.03%. Batch nhỏ tạo nhiều update hơn mỗi epoch, nên không coi đây là so sánh cùng số bước.

Kết quả mục 4.4 — softmax_stability

```
              logits                                    naive
stable
   [100.0, 101.0, 102.0]                                [nan, nan, nan]
[0.09003057330846786, 0.2447284758090973, 0.6652409434318542]
[-101.0, -102.0, -103.0] [0.6666666865348816, 0.2666666805744171, 0.06666667014360428]
[0.6652409434318542, 0.2447284758090973, 0.09003057330846786]
```

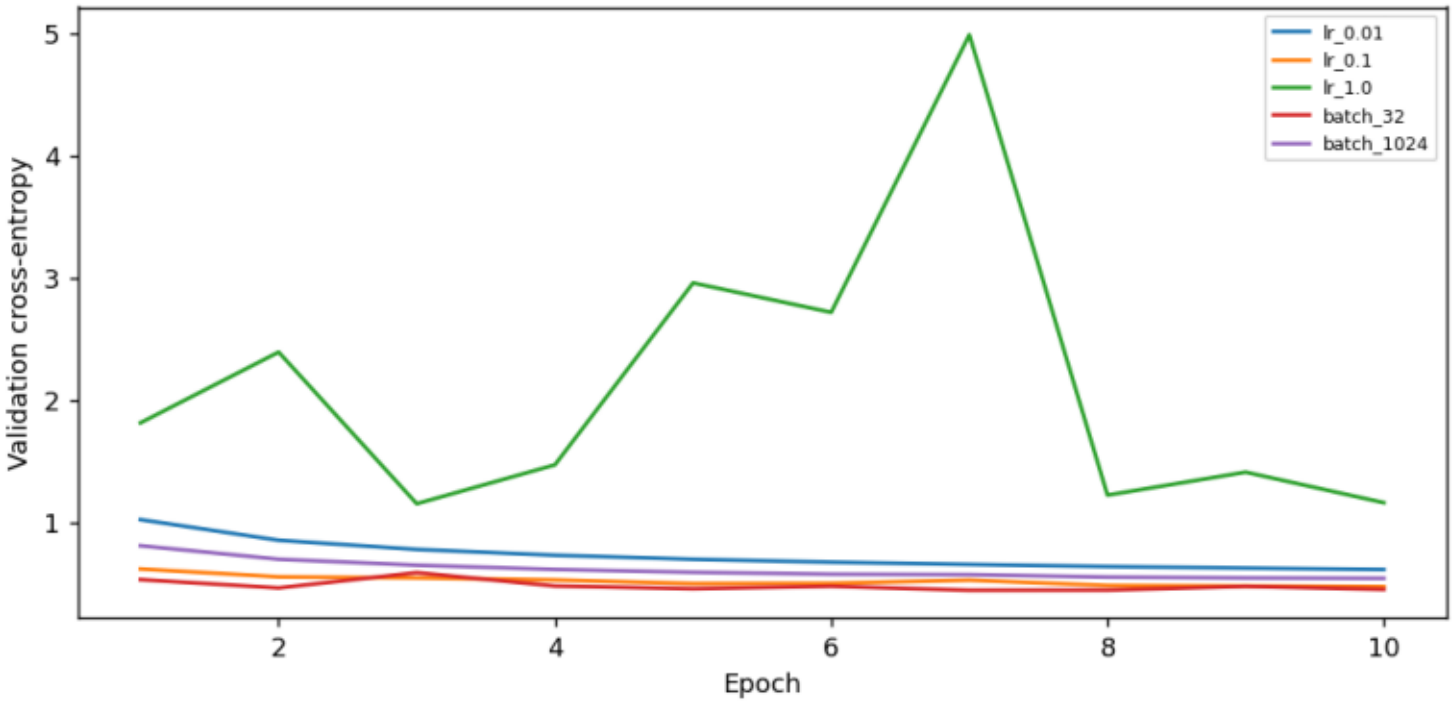
Kết quả mục 4.4 — cross_entropy_speed

implementation	seconds_per_call
dense_one_hot	0.000117599
index_fused	6.8769e-05

Kết quả mục 4.4 — comparison

name	hidden	epoch	lr	train_loss	val_loss	train_accuracy	val_accuracy
lr_0.01	()	10	0.01	0.600673	0.621915	0.8083	0.7949
lr_0.1	()	10	0.1	0.447767	0.482206	0.847367	0.8334
lr_1.0	()	10	1	1.02999	1.16873	0.844633	0.8268
batch_32	()	10	0.1	0.401198	0.459343	0.862133	0.8403
batch_1024	()	10	0.1	0.524815	0.550116	0.82495	0.8129

validation_learning_curves



Lê Sơn Trường — Chương 4, mục 4.5

4.5 — Concise Softmax (4 câu)

Câu 1

Nếu exp trả về số chuẩn hóa hữu hạn, khoảng x nằm xấp xỉ
[log(finfo.tiny), log(finfo.max)]. Khi tính cả subnormal, cận dưới lý thuyết
là log(tiny*eps). Các mốc không phải ngưỡng làm tròn tuyệt đối chính xác;
hardware có thể flush subnormal thành 0. Notebook xuất bảng FP64/FP32/BF16/FP16,
đồng thời tìm ngưỡng bằng torch.exp thật trên CPU và kiểm tra số biểu diễn kề
trước/kề sau bằng nextafter. Bảng "measured" là ngưỡng của môi trường chạy,
không nhầm log(min_normal) với ngưỡng bắt đầu underflow về đúng 0.
TF32 giữ exponent FP32, giảm mantissa cho một số CUDA matmul, không phải dtype
CPU exp riêng. Số nguyên 8 bit không có khoảng exp dấu phẩy động tương đương;
phải quy định quantization/scale và accumulator.

Câu 2

Khoảng 1…255 trong để tương ứng mã unsigned 8 bit bỏ 0, không phải signed INT8
thông thường (−128…127). Mở rộng dynamic range bằng scale/zero-point,
block floating point hoặc log quantization; đánh đổi độ phân giải, không tạo thêm
thông tin. Với log coding, nhân gần thành cộng mã, nhưng cộng giá trị cần log-add;
phép cộng/nhân chuẩn trên mã không còn trực tiếp đúng với giá trị biểu diễn.

Câu 3

Notebook tăng bản concise lên 20 epoch từ baseline 10. Validation accuracy có thể
giảm do overfitting hoặc dao động tối ưu; không khẳng định chắc chắn sẽ giảm ở
mọi lần chạy. Dùng early stopping theo validation, regularization/augmentation,
LR schedule. Không dùng nhãn test độc lập để quyết định dừng.

Câu 4

Đối chiếu các LR curve tại 4.4: LR nhỏ có thể tốt khi học đủ lâu nhưng chậm;
LR quá lớn dễ dao động/phân kỳ. Lưới tối ưu phụ thuộc epoch, batch và mô hình.
Để tránh lặp cùng thí nghiệm, mục này dùng lại kết quả LR đã lưu ở 4.4 và có
thêm bản concise 20 epoch riêng.

Ghi nhận chạy thật — mục 4.5

LR curves reuse section 4.4, avoiding duplicate runs. FP16/BF16 subnormals may be flushed to zero by hardware. TF32 uses FP32 exponent but is a CUDA matmul mode, not CPU exp dtype.

Kết quả mục 4.5 — exp_dynamic_range

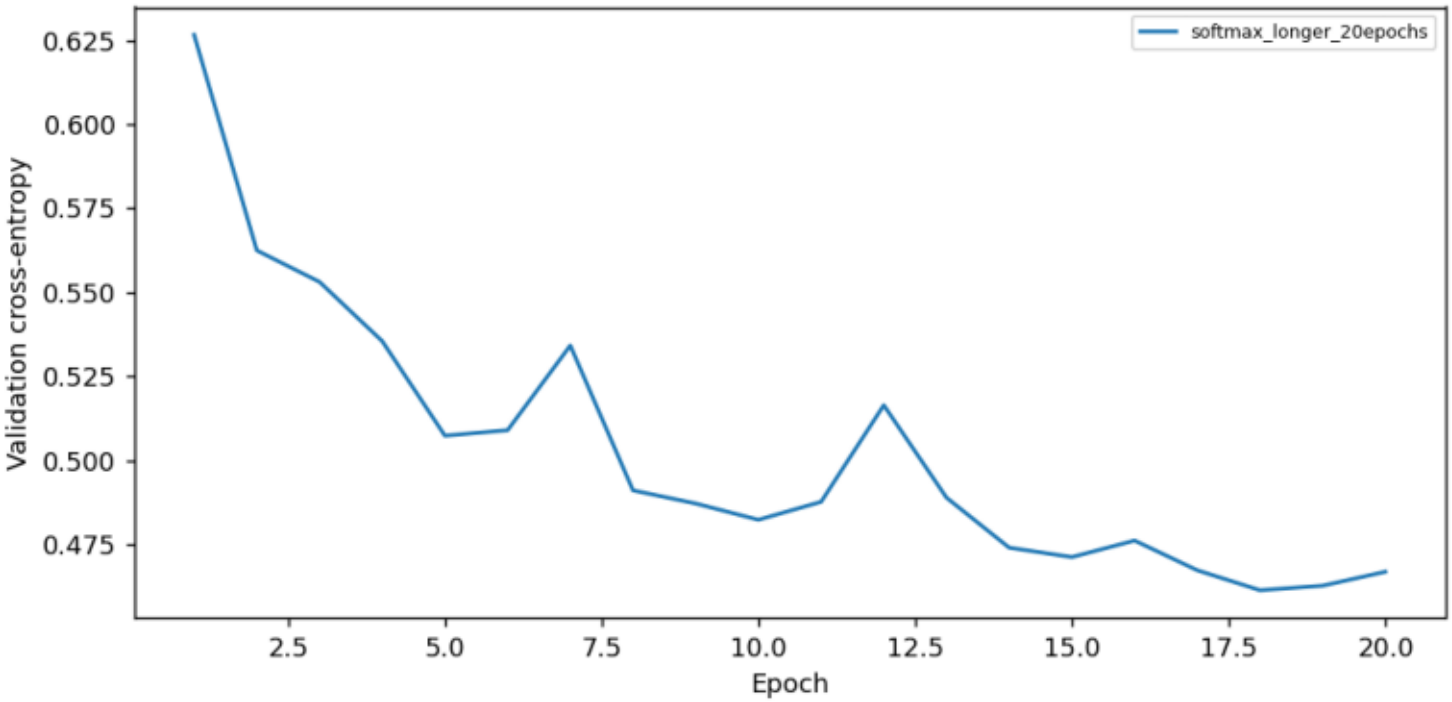
dtype	log_max_finite	log_min_normal	log_min_subnormal_theoretical
torch.float64	709.783	-708.396	-744.44
torch.float32	88.7228	-87.3365	-103.279
torch.bfloat16	88.7189	-87.3365	-92.1886
torch.float16	11.0899	-9.70406	-16.6355

Kết quả mục 4.5 — exp_measured_boundaries_cpu

dtype	smallest_representable_input_with_nonzero_exp	largest_representable_input_with_finite_exp	exp_at_previous_input	exp_at_next_input
torch.float64	-745.133	709.783	0	overflow
torch.float32	-103.972	88.7228	0	overflow
torch.bfloat16	-92.5	88.5	0	overflow
torch.float16	-17.3281	11.0859	0	overflow

Kết quả mục 4.5 — comparison

name	hidden	epoch	lr	train_loss	val_loss	train_accuracy	val_accuracy
softmax_longer_20epochs	()	20	0.1	0.425136	0.466817	0.852167	0.8314


softmax_longer_20epochs
Validation cross-entropy
0.625
0.600
0.575
0.550
0.525
0.500
0.475
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Epoch

Lê Sơn Trường — Chương 4, mục 4.6

4.6 — Generalization in Classification (4 câu)

Câu 1

Với classifier cố định, 0–1 error trên mẫu IID, Hoeffding cho
$P(|\varepsilon hat - \varepsilon| \geq \delta) \leq 2\exp(-2n\delta^2)$. Đặt $\delta = 0.0001$, failure $= 0.001$:
$n \geq \log(2000)/(2 \cdot 10^{-8})$, lấy trần 380.045.123 là cận đủ (điều kiện
"lớn hơn 99,9%" có thể chọn thêm một mẫu nếu muốn bất đẳng thức nghiêm).
Không phải cỡ mẫu cần thiết tối thiểu trong mọi phân phối.

Câu 2

Với nhãn nhị phân và oracle trả exact error, hỏi một mô hình luôn dự đoán 0,
rồi N mô hình mỗi mô hình chỉ đổi dự đoán ở một x_i. Error giảm 1/N nghĩa là
$y_i = 1$, tăng 1/N nghĩa là $y_i = 0$. Vậy N+1 lần gọi đủ; không khẳng định là tối thiểu.
Đa lớp: baseline lớp 0 và lần lượt đổi sang từng lớp khác, tối đa $1 + N(K-1)$
lần theo cách đơn giản. Nếu oracle làm tròn/giới hạn truy vấn thì lập luận không còn trực tiếp áp dụng.

Câu 3

Với classifier 1D sign(p(x)) và p bậc không quá 5, VC dimension=6.
Sáu điểm có thứ tự có tối đa 5 lần đổi dấu, đặt nghiệm giữa những lần đổi dấu
để thực hiện mọi nhãn. Bảy điểm nhãn luân phiên cần 6 nghiệm nên không shatter được.
Nếu "đa thức bậc 5" là đa biến thì cần chỉ rõ số biến, kết quả khác.

Câu 4

Rectangle song song trục trong R^2 có VC dimension=4. Bốn điểm trái/phải/
trên/dưới hình thoi có thể shatter. Với 5 điểm bất kỳ, chọn tối đa 4 điểm đạt
các cực trị $x_{min}, x_{max}, y_{min}, y_{max}$; còn ít nhất một điểm nằm trong bounding box
của các điểm đó. Gán các cực trị positive và điểm còn lại negative thì rectangle
không thực hiện được. Điểm trùng càng không tăng khả năng shatter.

Ghi nhận chạy thật — mục 4.6

Hoeffding sufficient n=380,045,123 for epsilon=0.0001, delta=0.001, fixed classifier and IID test draws.

Exact-error test oracle: baseline+N one-coordinate queries recovered labels [0, 1, 1, 0, 1, 0].

Lê Sơn Trường — Chương 4, mục 4.7

4.7 — Environment and Distribution Shift (4 câu)

Câu 1

Đổi thuật toán xếp hạng làm thay đổi thứ người dùng thấy/click; người dùng đổi truy vấn, nhà quảng cáo đổi giá thầu/nội dung để thích ứng. Log mới do chính hệ thống tạo ra khác log cũ, có feedback/selection bias. Cần thử nghiệm và theo dõi chứ không giả định dữ liệu log luôn IID từ một phân phối cố định.

Câu 2

Notebook tạo hai phân phối covariate tổng hợp và fit domain classifier source=0, target=1; đánh giá AUC trên hold-out độc lập. AUC cao hơn 0.5 là bằng chứng phân biệt được hai miền với classifier đó. AUC gần 0.5 không chứng minh hai phân phối hoàn toàn giống nhau: có thể detector yếu hoặc dữ liệu ít.

Câu 3

Nếu $p_s(y|x)=p_t(y|x)$, support có chồng lấp và priors miền bằng nhau, $w(x)=p_t(x)/p_s(x)=D(x)/(1-D(x))$. Tổng quát phải nhân thêm prior source/target. Notebook dùng trọng số trên source hold-out để ước lượng mean ở target, so sánh mean không trọng số, trọng số và target mean thật. Clip weights giảm variance nhưng thêm bias; xuất effective sample size. Không dùng cách này cho mọi concept shift.

Câu 4

Cỡ mẫu hữu hạn, phụ thuộc giữa mẫu, nhãn sai, đo đạc sai, selection bias, model-selection/multiple testing và leakage đều ảnh hưởng empirical risk. Ngay khi không có shift, train error vẫn có thể lạc quan do thuật toán tối ưu trực tiếp trên train.

Ghi nhận chạy thật — mục 4.7

Synthetic covariate shift with shared conditional y=x1+x2. Domain priors are equal; weights=p(target|x)/p(source|x). Correction needs overlap and unchanged p(y|x), not label/concept shift.

Kết quả mục 4.7 — shift_detection_correction

heldout_domain_auc	source_mean	importance_weighted_mean	target_mean	effective_sample_size
0.788305	-0.0245856	1.55283	1.58272	377.953

Lê Sơn Trường — Chương 5, mục 5.1

5.1 — Multilayer Perceptrons (7 câu)

Câu 1

Ghép các affine không activation cho $W_L \dots W_1 x + b_{eff}$, vẫn là affine. Ví dụ đầu vào/ra 2D, tầng giữa chỉ 1 chiều thì hạng tích ≤ 1, không biểu diễn được ánh xạ identity hạng 2. Nhiều tầng tuyến tính không tăng lớp hàm, có thể giảm nó khi có bottleneck, dù tham số hóa ảnh hưởng cách tối ưu.

Câu 2

$\mathrm{pReLU}(x) = x$ nếu $x \ge 0$, αx nếu $x < 0$. Đạo hàm theo x là 1 ở $x > 0$, α ở $x < 0$; tại 0 không khả vi khi $\alpha \ne 1$ (implementation chọn một giá trị quy ước). Nếu α học được, $\partial \mathrm{pReLU}/\partial \alpha = x$ khi $x < 0$ và 0 khi $x > 0$.

Câu 3

$\mathrm{Swish} = x \cdot \mathrm{sigmoid}(\beta x)$, đặt $s = \mathrm{sigmoid}(\beta x)$: $d/dx = s + \beta x\, s(1-s)$. Nếu β là tham số học: $\partial/\partial\beta = x^2 s(1-s)$.

Câu 4

Affine liên tục và tuyến tính từng miền; ReLU/pReLU liên tục và affine từng miền. Ghép hữu hạn các hàm này cho hàm liên tục piecewise-affine trên các miền đa diện (sách gọi piecewise-linear; có bias thì chính xác hơn là affine). Số miền có thể tăng khi thêm tầng, nhưng không tạo gián đoạn tại ranh giới activation.

Câu 5

Từ định nghĩa: $\tanh x = (e^{2x}-1)/(e^{2x}+1) = 2\,\mathrm{sigmoid}(2x) - 1$. Do đó $\tanh(Wx+b) = 2\,\mathrm{sigmoid}(2Wx+2b) - 1$. Hệ số 2 và -1 được hấp thụ vào trọng số/bias của tầng affine tiếp theo; chiều ngược dùng sigmoid $z = (\tanh(z/2)+1)/2$. Với activation ở tầng ẩn và đầu ra affine, hai lớp hàm biểu diễn tương đương sau tái tham số hóa; điều này không bảo đảm dynamics tối ưu giống nhau.

Câu 6

Nonlinearity phụ thuộc batch (như batch normalization khi train) làm đầu ra của một mẫu phụ thuộc các mẫu đi cùng, kích thước batch và thống kê batch. Batch quá nhỏ hoặc train/eval khác phân phối có thể gây nhiễu/lệch thống kê. Phải xử lý running statistics và chế độ eval đúng; trong dữ liệu chuỗi cần tránh thông tin tương lai lọt vào thống kê dùng cho dự đoán quá khứ.

Câu 7

$\sigma'(x) = \sigma(x)(1-\sigma(x)) \to 0$ khi $|x|$ lớn, ví dụ $x = \pm 30$. Nhiều tầng sigmoid nhân các gradient nhỏ có thể làm gradient về đầu mạng rất nhỏ. Notebook đo gradient tại $-30, -10, 0, 10, 30$; FP32 ở phía bão hòa dương còn có thể làm tròn về đúng 0.

Ghi nhận chạy thật — mục 5.1

tanh(x)+1=2 sigmoid(2x) verified numerically.

Kết quả mục 5.1 — sigmoid_gradients

x	gradient
-30	9.35762e-14
-10	4.53958e-05
0	0.25
10	4.54167e-05
30	0

Lê Sơn Trường — Chương 5, mục 5.2

5.2 — Implementation of MLP (9 câu)

Câu 1

Baseline sách: 784→256→10, ReLU, SGD LR=0.1, batch=256, 10 epoch.
Notebook khảo sát 1,64,128,256,512 neuron trên cùng seed, vẽ accuracy theo độ rộng.
“Tốt nhất” chỉ là tốt nhất trong lưới này với ngân sách 10 epoch, không phải
siêu tham số tối ưu toàn cục. Kết quả cuối mục được tính tự động từ CSV.

Câu 2

So sánh baseline một tầng 256 với hai tầng 256→256, giữ LR, batch, epoch.
Thêm tầng tăng khả năng kết hợp phi tuyến và chi phí, nhưng không bảo đảm tốt hơn;
bảng/learning curve trong notebook cho biết kết quả thực tế.

Câu 3

Một neuron ẩn là bottleneck một chiều: nhiều đầu vào bị nén về một số.
Sau ReLU có thể chết hẳn nếu preactivation âm cho mọi mẫu. Nhiệm vụ có nhiều
lớp/đặc trưng có thể mất thông tin quan trọng, nên đây là lựa chọn quá hạn chế.
Không nói một neuron không thể giải mọi bài toán: một bài rất đơn giản vẫn có thể đủ.

Câu 4

Giữ width=256, so LR=0.01/0.1/0.5 với 10 epoch. LR nhỏ cần nhiều epoch hơn;
LR lớn có thể hội tụ nhanh hoặc dao động. Chọn theo validation loss/accuracy và
đại lượng mục tiêu; không suy ra LR=0.5 tốt cho mọi kiến trúc chỉ từ một lần đo.

Câu 5

Thử thêm cấu hình kết hợp: hai tầng 128 với LR=0.1, 15 epoch; hai tầng 256 với
LR=0.05, 15 epoch, rồi so với các cấu hình trước. Báo cáo cấu hình có val loss
thấp nhất và accuracy cao nhất trong tập đã đo, không tuyên bố tối ưu toàn bộ không gian.
Với k lựa chọn ở mỗi d tham số, grid đầy đủ có k^d lần fit; các yếu tố còn tương tác.
Chiến lược hiệu quả: random search trên log LR/weight decay, ngân sách ngắn cho
nhiều cấu hình rồi successive halving/ASHA, tăng ngân sách cho ứng viên tốt;
giữ test riêng nếu cần đánh giá cuối sau tuning.

Câu 6

Notebook có bản ScratchMLP với W1,b1,W2,b2 và matmul/ReLU rõ ràng, cùng bản
nn.Sequential. Để đo công bằng, copy cùng trọng số, cùng batch và loss,
warm-up rồi đo forward+backward 20 lần ở width 64/256/512. Bảng timing này
tách khỏi tổng thời gian học, vì hai bản học gốc có cách khởi tạo khác nhau.
Tăng độ phức tạp có thể làm GEMM/kernel chi phối, giảm tỷ lệ overhead Python;
không khẳng định luôn framework nhanh hơn mà không xem số đo.

Câu 7

Notebook đo matmul CPU ở 1024/1025/1026/1028/1032, lấy median sau warm-up.
Kích thước chia hết tile/SIMD có thể nhanh hơn nhưng phụ thuộc BLAS/kernel,
cache và hệ thống; không so chỉ thời gian mà quên số phép toán thay đổi theo n^3.

Máy có Ryzen 9 7945HX và RTX 4060 Laptop; môi trường hiện tại cài PyTorch CPU,
torch.cuda.is_available() là False nên chưa có số đo GPU. Không giả lập
GPU timing thành kết quả thật; benchmark_gpu.py chạy phép đo cùng kích thước
khi dùng kernel PyTorch hỗ trợ CUDA, có synchronize trước/sau timer.

RAM báo cáo mỗi module DataWidth=64 bit; CPU hỗ trợ hai kênh theo AMD (https://www.amd.com/en/products/processors/laptop/ryzen/7000-series/amd-ryzen-9-7945hx.html), nên bề rộng tổng lý thuyết 2×64=128 bit nếu hai kênh hoạt động. DDR5 có thể chia mỗi DIMM thành các subchannel; không đồng nhất AddressWidth=64 với memory bus. RTX 4060 Laptop có giao tiếp VRAM 128 bit theo NVIDIA (https://www.nvidia.com/en-ph/geforce/laptops/compare/). Không dùng WMI AdapterRAM để suy ra dung lượng VRAM vì trường đó có giới hạn.

Câu 8

Khảo sát ReLU/tanh/sigmoid trên mạng một tầng 256 neuron với cùng LR=0.1, batch 256, 10 epoch. ReLU tránh bão hòa phía dương nhưng có dead neurons; sigmoid/tanh có thể bão hòa. Lấy kết luận theo bảng cụ thể; khác LR hoặc khởi tạo có thể thay thứ tự tốt/xấu.

Câu 9

So zero, normal std=0.01, Xavier và default của Linear, cùng dữ liệu/seed. Zero init ở MLP gây đối xứng; với ReLU tại 0, gradient tầng đầu có thể bằng 0, chỉ bias đầu ra học prior lớp. Xavier/Kaiming giúp kiểm soát phương sai nhưng cần khớp activation. Trong linear/softmax không tầng ẩn, zero init không gây cùng dạng đối xứng neuron ẩn; xem mục 5.4.

Ghi nhận chạy thật — mục 5.2

CUDA available=False. No GPU timing or physical memory-bus width is invented. CPU word size 64bit is not memory bus width; see hardware.json.

From-scratch vs framework speeds above include initialization differences. A separate equal-weight forward/backward benchmark follows.

Kết luận thực nghiệm — mục 5.2

Nhận xét từ số đo: trong nhóm độ rộng, width_512 có accuracy cao nhất (85.51%). Trong toàn bộ cấu hình thử, lr_0.5 có loss thấp nhất (0.3704), accuracy 86.87%. Zero init và bottleneck 1 neuron chỉ đạt 10% trong lần chạy này. Không suy rộng thành siêu tham số tối ưu toàn cục.

Kết quả mục 5.2 — comparison

name	hidden	epoch	lr	train_loss	val_loss	train_accuracy	val_accuracy
width_1	(1,)	10	0.1	2.3026	2.3026	0.1	0.1
width_64	(64,)	10	0.1	0.381817	0.423548	0.865217	0.8488
width_128	(128,)	10	0.1	0.368949	0.412845	0.869767	0.8516
width_256	(256,)	10	0.1	0.367504	0.411041	0.869967	0.852
width_512	(512,)	10	0.1	0.362927	0.406771	0.870767	0.8551
two_layers	(256, 256)	10	0.1	0.345295	0.394196	0.875433	0.8588
lr_0.01	(256,)	10	0.01	0.563067	0.584918	0.812133	0.7998
lr_0.5	(256,)	10	0.5	0.30136	0.370444	0.888367	0.8687
tanh	(256,)	10	0.1	0.381716	0.422478	0.863783	0.8454
sigmoid	(256,)	10	0.1	0.473978	0.501411	0.832317	0.8192
zero_init	(256,)	10	0.1	2.3026	2.3026	0.1	0.1
normal001_init	(256,)	10	0.1	0.375348	0.417258	0.866883	0.8494
xavier_init	(256,)	10	0.1	0.346882	0.396117	0.876783	0.8611
scratch_width256	(256,)	10	0.1	0.379101	0.420425	0.86535	0.85
joint_layers128_lr0.1_15epochs	(128, 128)	15	0.1	0.345123	0.400688	0.870783	0.8492
joint_layers256_lr0.05_15epochs	(256, 256)	15	0.05	0.381216	0.424355	0.862717	0.8451

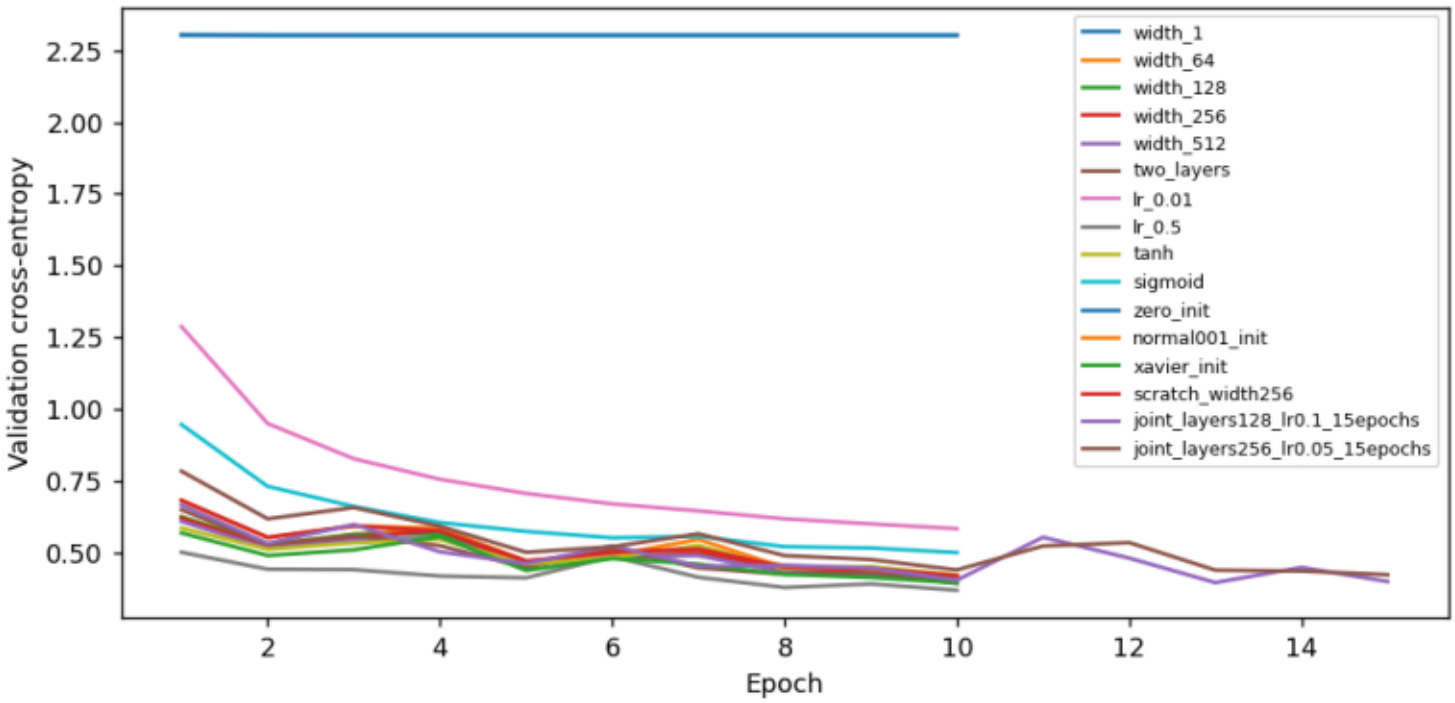
Kết quả mục 5.2 — matrix_alignment_cpu

size	cpu_seconds
1024	0.0153669
1025	0.0159556
1026	0.0161215
1028	0.0156255
1032	0.0152154

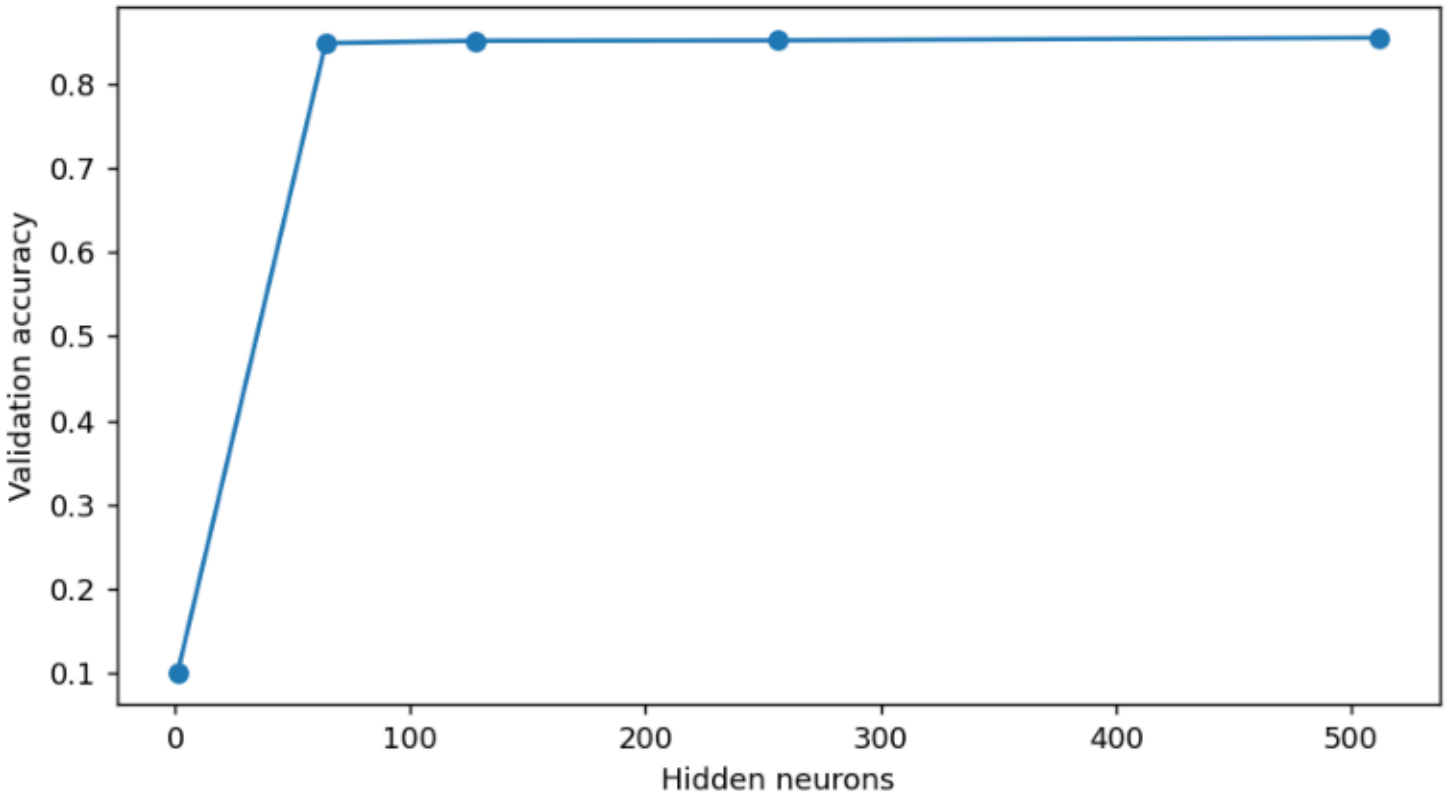
Kết quả mục 5.2 — equal_weights_speed

width	implementation	forward_backward_seconds
64	framework	0.00064443
64	scratch	0.000638125
256	framework	0.00179374
256	scratch	0.00185146
512	framework	0.00331014
512	scratch	0.00354193

validation_learning_curves



neurons_vs_accuracy



Lê Sơn Trường — Chương 5, mục 5.3

5.3 — Forward/Backward/Computational Graphs (5 câu)

Câu 1

Loss scalar f và X shape n×m ⇒ gradient ∂f/∂X cũng shape n×m,
mỗi phần tử là độ nhạy của f theo phần tử tương ứng của X.

Câu 2

Theo quy ước batch hàng: $Z=XW1+1b1^T$; $H=\varphi(Z)$; $O=HW2$;
$J=L(O,Y)+(\lambda/2)(\|W1\|^2_F+\|W2\|^2_F)$, không phạt b1.
Đồ thị ở notebook: X,W1,b1→Z→H; H,W2→O; O,Y→L; W1,W2→R; L,R→J.

Đặt $G=\partial L/\partial O$; với softmax-CE mean, $G=(\text{softmax}(O)-\text{onehot}(Y))/N$.
Backward:

- $\partial J/\partial W2=H^TG+\lambda W2$.
- $D=(GW2^T)\odot\varphi'(Z)$.
- $\partial J/\partial W1=X^TD+\lambda W1$; $\partial J/\partial b1=\Sigma_{\text{hàng}}\ D$; $\partial J/\partial X=DW1^T$.

Với vector cột, transpose các ma trận theo quy ước của sách; không trộn hai quy ước.

Câu 3

Input d, hidden h, output q, batch N: mô hình gốc không bias có $P_0=dh+hq$;
phiên bản thêm hidden bias ở câu 2 có $P=P_0+h$ tham số.
FP32 tham số chiếm 4P byte, gradient thêm 4P; SGD thuần không có momentum state,
Adam thêm khoảng 8P cho hai moment. Các activation X,Z,H,O có khoảng
N(d+2h+q) phần tử lưu cho backward, thêm bộ nhớ loss/workspace/mask tùy framework.
Ví dụ FP32, d=784,h=256,q=10,N=32: mô hình gốc có 203.264 tham số
(813.056 byte), mô hình thêm bias có 203.520 tham số (814.080 byte).
Prediction không giữ graph/gradient, cần tham số và activation đang dùng; peak
phụ thuộc tái sử dụng buffer, không thể cho một số byte duy nhất nếu chưa chỉ rõ
batch,dtype,optimizer và implementation.

Câu 4

Phải giữ graph của gradient (create_graph=True); graph mới gồm cả phép tính
backward. Hessian đầy đủ có P^2 phần tử và thường cần nhiều lượt, tốn hơn bậc một.
HVP chỉ cần directional derivative của gradient, tránh materialize Hessian.
Notebook kiểm tra $f=\Sigma x^3$ có $H\cdot 1=6x$.

Câu 5

Có thể partition theo tầng/pipeline/model parallel qua nhiều GPU, truyền
activation và gradient giữa thiết bị. Ưu điểm: mô hình lớn hơn VRAM một GPU;
nhược: communication, pipeline bubble, đồng bộ và triển khai phức tạp.
Giảm minibatch đơn giản hơn và giảm activation memory, nhưng không giảm kích
thước tham số/optimizer state; model parallel/sharding cần khi phần này đã quá lớn.

Ghi nhận chạy thật — mục 5.3

X shape=(5, 3), gradient shape=(5, 3), b1 gradient shape=(4,)

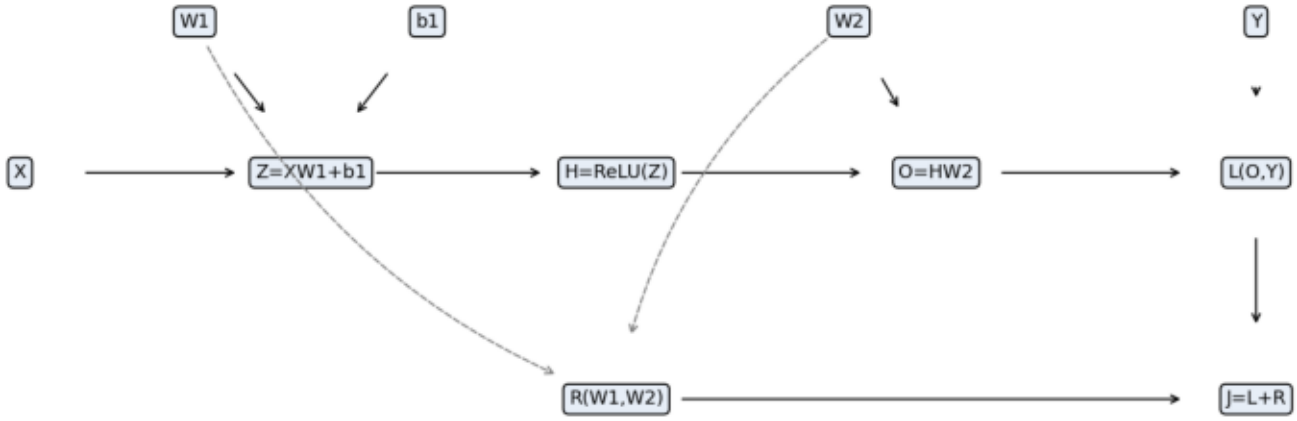
Graph: X,W1,b1 -> Z -> ReLU -> H; H,W2 -> O; O,Y -> loss; weight matrices -> L2 -> objective.

Second-derivative Hessian-vector product=[6.0, 12.0]

Kết quả mục 5.3 — memory_accounting

model	parameters	parameter_bytes_fp32	parameter_plus_gradient_bytes_fp32	stored_X_Z_H_O_bytes_fp32
original_no_bias	203264	813056	1626112	167168
hidden_bias_added	203520	814080	1628160	167168

computational_graph



Lê Sơn Trường — Chương 5, mục 5.4

5.4 — Numerical Stability and Initialization (4 câu)

Câu 1

Các filter convolution giống nhau trong cùng nhóm hoặc attention heads có toàn bộ tham số/đường đi giống nhau có thể nhận gradient giống nhau, học trùng chức năng. Các nhánh residual song song khởi tạo giống nhau cũng có đối xứng tương tự. Phá bằng khởi tạo ngẫu nhiên độc lập hoặc thiết kế bất đối xứng có chủ đích.

Câu 2

Linear regression có thể khởi tạo cùng giá trị, kể cả 0. Softmax cũng có thể zero: nhãn từng lớp tạo gradient khác nhau $X^T(p-y)$ nên các cột không nhất thiết giữ giống nhau. Vấn đề đối xứng nặng là nhiều neuron ẩn đồng vai trò cùng tham số, không phải mọi mô hình có nhiều tham số đều cấm zero init.

Câu 3

Không có quy tắc nhân riêng eigenvalue cho hai ma trận bất kỳ. Ví dụ hai ma trận tam giác $A=[[1,M],[0,1]]$, $B=A^T$ đều có eigenvalue 1, nhưng AB có eigenvalue lớn cỡ M^2. Notebook kiểm tra $M=100$.

Với spectral norm/singular value: $\sigma_{max}(AB) \le \sigma_{max}(A)\sigma_{max}(B)$; với ma trận vuông khả nghịch $\sigma_{min}(AB) \ge \sigma_{min}(A)\sigma_{min}(B)$, nên $\kappa_2(AB) \le \kappa_2(A)\kappa_2(B)$. Gradient qua nhiều tầng chứa tích Jacobian: σ lớn gây explode, nhỏ gây vanish. Khởi tạo tốt cần kiểm soát singular values/phương sai và activation, không chỉ eigenvalue.

Câu 4

LARS dùng local rate gần $\eta_l = \eta \cdot \tau \|w_l\| / (\|g_l\| + \beta\|w_l\| + \varepsilon)$ để scale bước cập nhật tương đối theo norm tầng; xem You, Gitman, Ginsburg 2017 (https://arxiv.org/abs/1708.03888). Có thể dùng giảm LR/warm-up, clipping, normalization và khởi tạo phù hợp để phòng divergence. Nếu tham số đã thành Inf/NaN, scale sau đó không khôi phục thông tin; cần phát hiện trước update hoặc quay lại checkpoint hợp lệ. LARS không bảo đảm sửa mọi dạng bất ổn và không phải một phép “chữa” NaN sau khi mất dữ liệu.

Ghi nhận chạy thật — mục 5.4

Eigenvalues A=[1.0, 1.0], B=[1.0, 1.0], AB=[10001.999900019995, 9.998000499855664e-05]

Individual eigenvalues do not control a non-normal product; singular-value bounds are the safe general gradient bounds.

Lê Sơn Trường — Chương 5, mục 5.5

5.5 — Generalization in Deep Learning (5 câu)

Câu 1

Mạng quá tham số có thể fit nhãn ngẫu nhiên nhưng vẫn tổng quát hóa trên dữ liệu có cấu trúc. Capacity worst-case theo số tham số/VC thường cho cận quá lỏng, bỏ qua dữ liệu, margin/norm, nghiệm và implicit bias của thuật toán tối ưu. Không có nghĩa complexity vô nghĩa, chỉ các thước đo thô chưa giải thích đầy đủ.

Câu 2

Early stopping hạn chế thời gian tối ưu/độ phức tạp hữu hiệu của nghiệm, ngăn mô hình tiếp tục fit nhiễu train. Với một số bài toán tuyến tính nó có liên hệ spectral regularization, nhưng không đồng nhất mọi early stopping với cùng một L2.

Câu 3

Theo dõi validation metric định trước, lưu best checkpoint và dừng sau patience khi không cải thiện đủ min_delta. Chọn metric/độ kiên nhẫn trước khi nhìn test. Notebook chọn epoch min validation loss từ một đường học đã đo, không giả định epoch đó luôn có accuracy cao nhất.

Câu 4

Lợi ích lớn hơn khi có khoảng cách train/validation, nhãn nhiễu hoặc mô hình có khả năng ghi nhớ và validation bắt đầu xấu đi trong lúc train còn tốt lên. Nếu cả hai đang cải thiện hoặc đã plateau giống nhau, early stopping có thể không cải thiện accuracy; phải xem dữ liệu và đường học cụ thể.

Câu 5

Tiết kiệm thời gian, điện và chi phí tính toán; dùng ngân sách cho nhiều cấu hình hơn. Lợi ích tài nguyên vẫn tồn tại ngay khi generalization không tăng.

Ghi nhận chạy thật — mục 5.5

Checkpoint is selected from a measured curve, not assumed to outperform final training. The curve is softmax from section 4.5, used only to demonstrate the criterion.

Kết quả mục 5.5 — early_stopping

selected_epoch	selected_val_loss	selected_val_accuracy	final_epoch	final_val_loss	final_val_accuracy
18	0.461317	0.8367	20	0.466817	0.8314

Lê Sơn Trường — Chương 5, mục 5.6

5.6 — Dropout (7 câu)

Câu 1

Giữ kiến trúc hai tầng 256→256, LR=0.1, batch256; so p=(0.2,0.5),
(0.5,0.2), baseline không dropout và cấu hình sách (0.5,0.5).
Bảng ghi accuracy/loss thật; các cặp hoán đổi chạy cùng 20 epoch.
Drop sớm hơn có thể làm mất tín hiệu đầu vào tầng sau nhiều hơn; không suy ra hoán đổi p luôn cho kết quả như nhau dù tổng p bằng nhau.

Câu 2

Hai cấu hình có dropout và baseline không dropout chạy 20 epoch, xuất kết quả ở epoch10 và epoch20 trên cùng run. Tăng epoch có thể giúp mạng dropout học bù nhiễu; cũng có thể baseline overfit. Kết luận theo số đo, không hứa dropout luôn thắng.

Câu 3

Inverted dropout $z=m\cdot h/(1-p)$, $m\sim\text{Bernoulli}(1-p)$. Với h cố định,
$E[z|h]=h$ và $\text{Var}(z|h)=p\cdot h^2/(1-p)$; không dropout thì mask variance=0.
Nếu H ngẫu nhiên độc lập mask:
$\text{Var}(Z)=\text{Var}(H)+p\cdot E[H^2]/(1-p)$.
Notebook đo empirical variance qua 1.024 ảnh cố định và toàn bộ units ở mỗi tầng, theo epoch. Eval và train được chạy riêng: đường eval tắt toàn bộ dropout; đường train bật dropout đúng như lúc học. Lưu/khôi phục RNG và mode quanh phép đo để không làm thay đổi mask của epoch tiếp theo. Đó không phải cùng đại lượng với variance theo mask khi giữ một activation h cố định; đồ thị ghi rõ cách đo.

Câu 4

Inference tiêu chuẩn tắt dropout vì inverted dropout đã giữ kỳ vọng khi train; để mask ngẫu nhiên làm dự đoán biến động không cần thiết. Ngoại lệ MC dropout cố ý giữ mask và lấy nhiều lần dự đoán để khảo sát bất định; phải thiết kế/đánh giá riêng.

Câu 5

So baseline, dropout, weight decay=1e−4 và cả hai trên cùng kiến trúc/20 epoch.
Hai cơ chế khác nhau: dropout tạo nhiễu activation, weight decay hạn chế trọng số.
Hiệu quả không nhất thiết cộng; kết hợp có thể lợi ích giảm dần hoặc regularize quá mạnh. Bảng định lượng trong notebook là căn cứ kết luận cho dữ liệu này.

Câu 6

Drop trực tiếp trọng số tạo $W\text{tilde}=M\odot W/(1-p)$ (DropConnect).
Các output units có mẫu nhiễu liên kết khác activation dropout; phải tính mask trên ma trận, tắt khi eval, kỳ vọng affine vẫn giữ nhưng kỳ vọng sau nonlinearity không nhất thiết bằng output mạng mean. Không coi hai kỹ thuật tương đương.

Câu 7

Thử thêm Gaussian noise mean0,std=0.15 vào mỗi tầng ẩn khi train, tắt lúc eval; giữ cùng kiến trúc/20 epoch với dropout. Notebook so sánh thật trên Fashion-MNIST. Nếu không thắng dropout thì báo đúng; câu hỏi mang tính nghiên cứu, không có bảo đảm một kỹ thuật mới tự nghĩ luôn vượt baseline. Có thể tuning std bằng validation, nhưng cần ngân sách và test độc lập để khẳng định cải thiện bền vững.

Ghi nhận chạy thật — mục 5.6

Noise/dropout can help or hurt; compare measured validation scores, no promise Gaussian noise wins. Eval and stochastic train variance are measured in separate forwards on 1024 fixed images. RNG and module modes are restored, so monitoring does not alter later training masks. Empirical variance across samples and units differs from conditional mask variance p/(1-p)*h^2.

Kết luận thực nghiệm — mục 5.6

Nhận xét từ số đo tại 20 epoch: không dropout 87.30%; p=(0.2,0.5) 87.12%; p=(0.5,0.2) 87.01%; weight decay 87.27%; kết hợp 87.06%; Gaussian noise 86.77%. Các mức regularization này chưa vượt baseline không dropout; không bịa kết luận cải thiện.

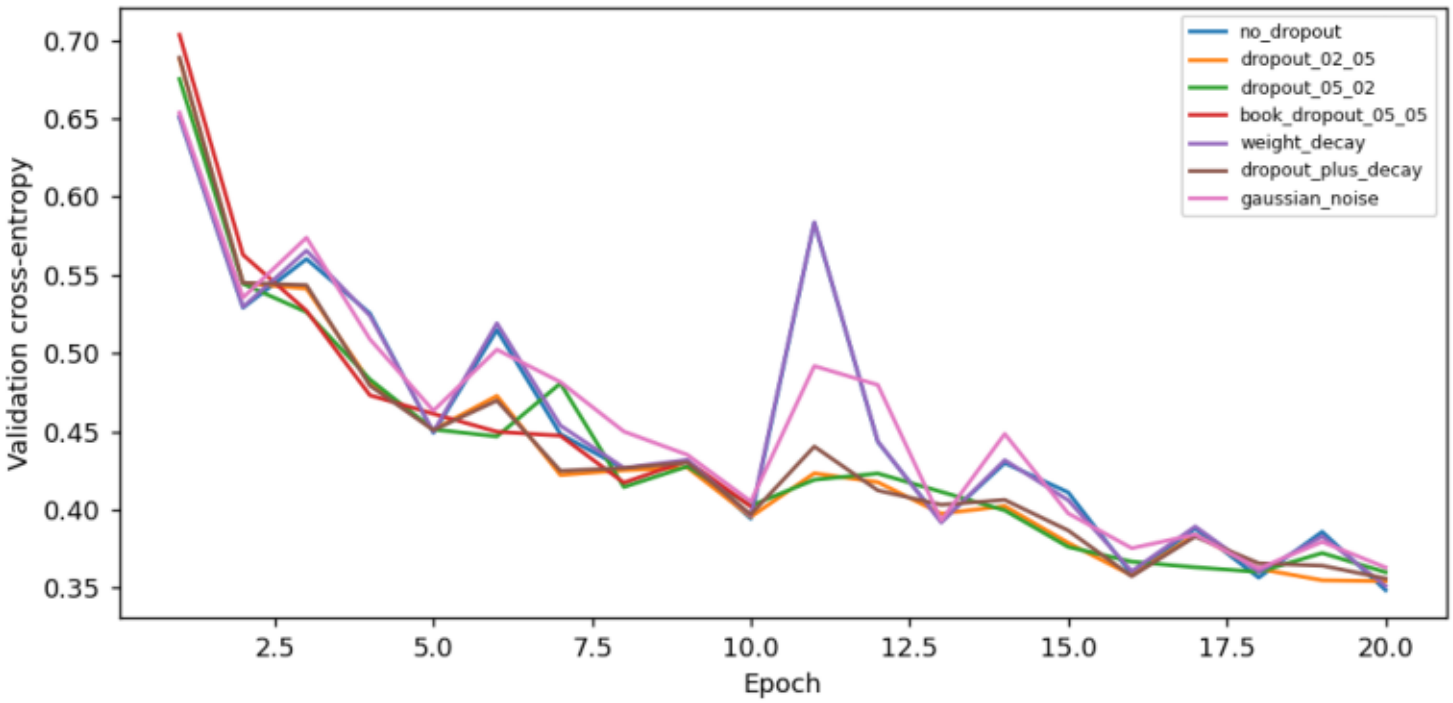
Kết quả mục 5.6 — comparison

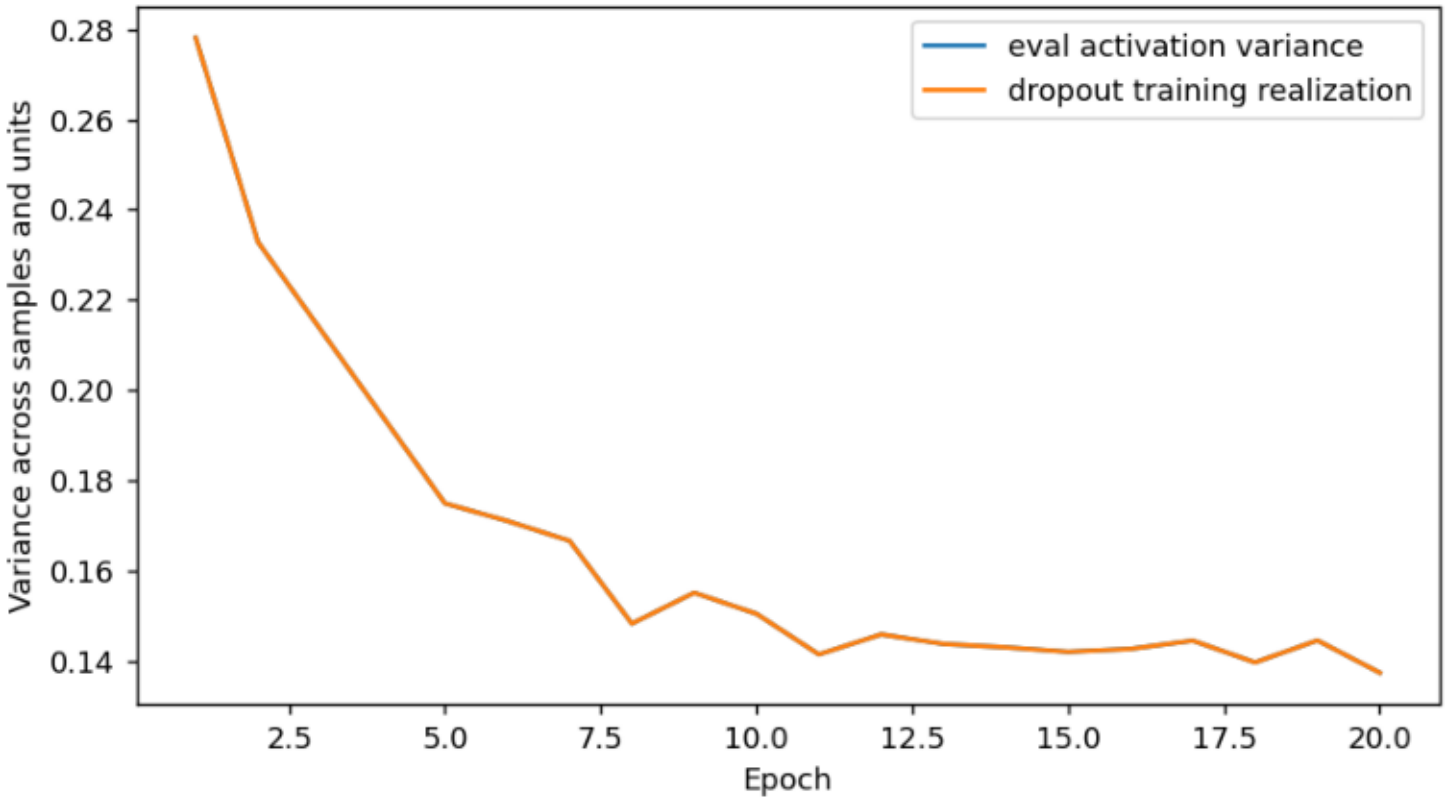
name	hidden	epoch	lr	train_loss	val_loss	train_accuracy	val_accuracy
no_dropout	(256, 256)	20	0.1	0.277244	0.348396	0.8971	0.873
dropout_02_05	(256, 256)	20	0.1	0.290213	0.354176	0.89295	0.8712
dropout_05_02	(256, 256)	20	0.1	0.298256	0.359903	0.889733	0.8701
book_dropout_05_05	(256, 256)	10	0.1	0.358442	0.401914	0.867283	0.8546
weight_decay	(256, 256)	20	0.1	0.282259	0.35111	0.895533	0.8727
dropout_plus_decay	(256, 256)	20	0.1	0.292897	0.355896	0.892367	0.8706
gaussian_noise	(256, 256)	20	0.1	0.300873	0.362939	0.887017	0.8677

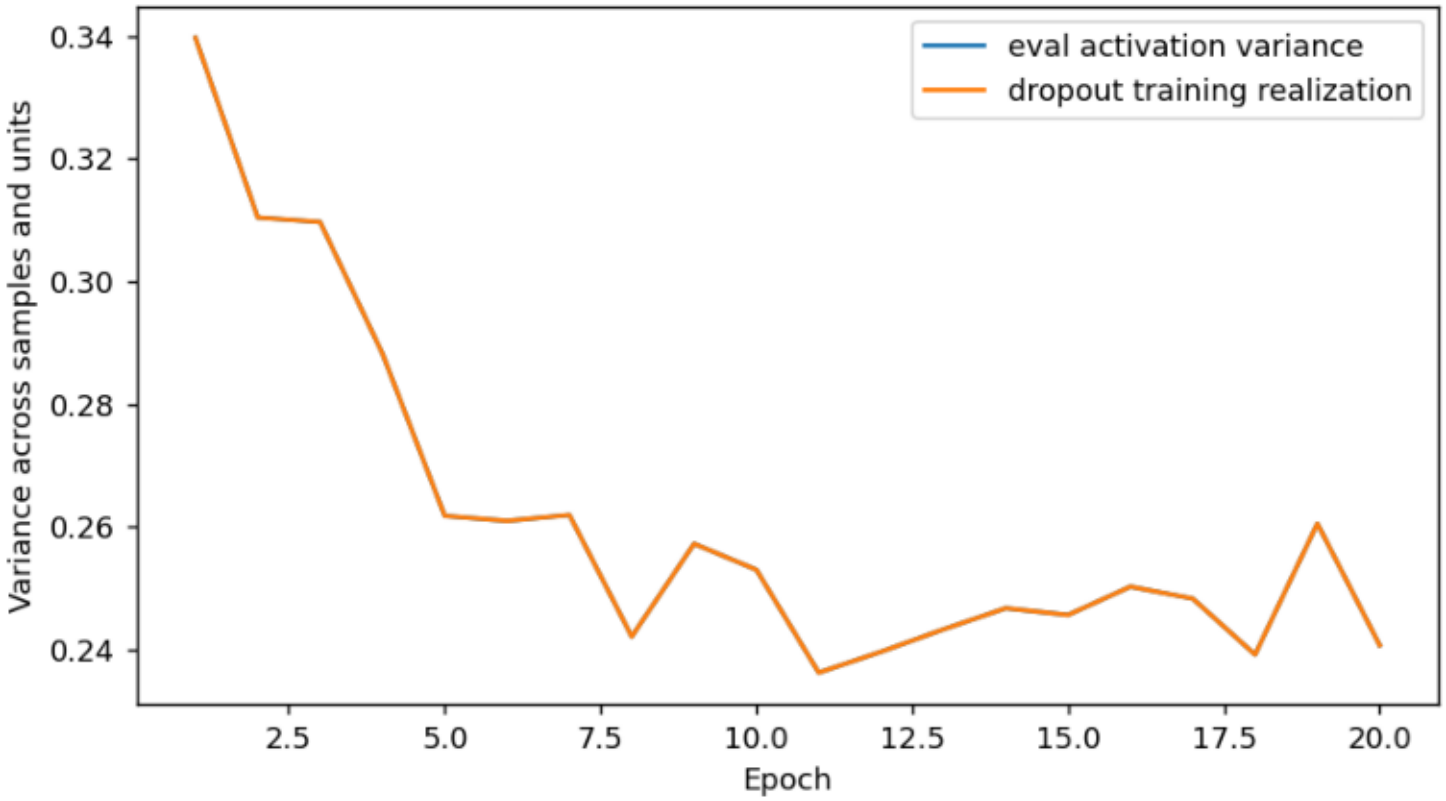
Kết quả mục 5.6 — longer_training

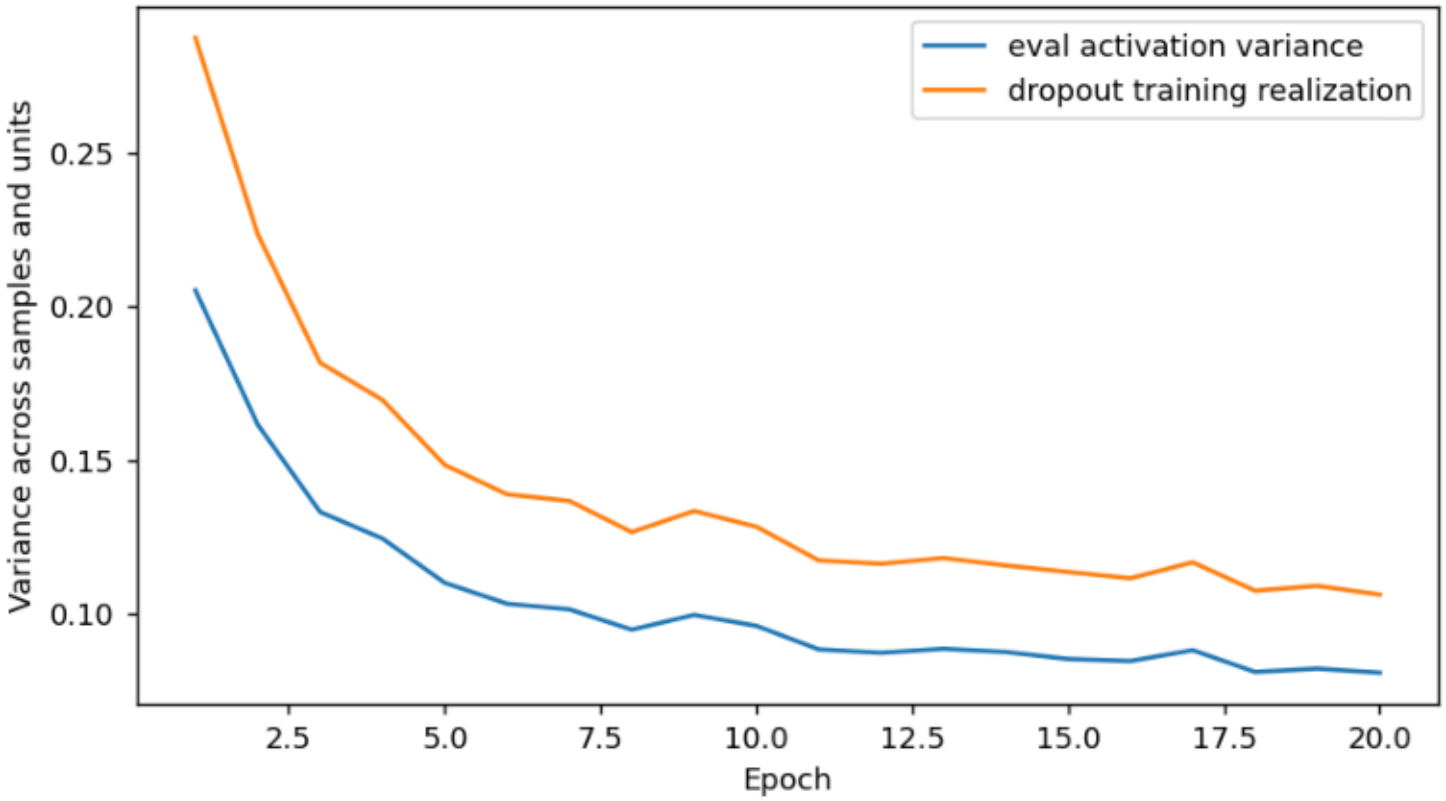
name	epoch10_accuracy	epoch20_accuracy
no_dropout	0.8588	0.873
dropout_02_05	0.8555	0.8712
dropout_05_02	0.8548	0.8701

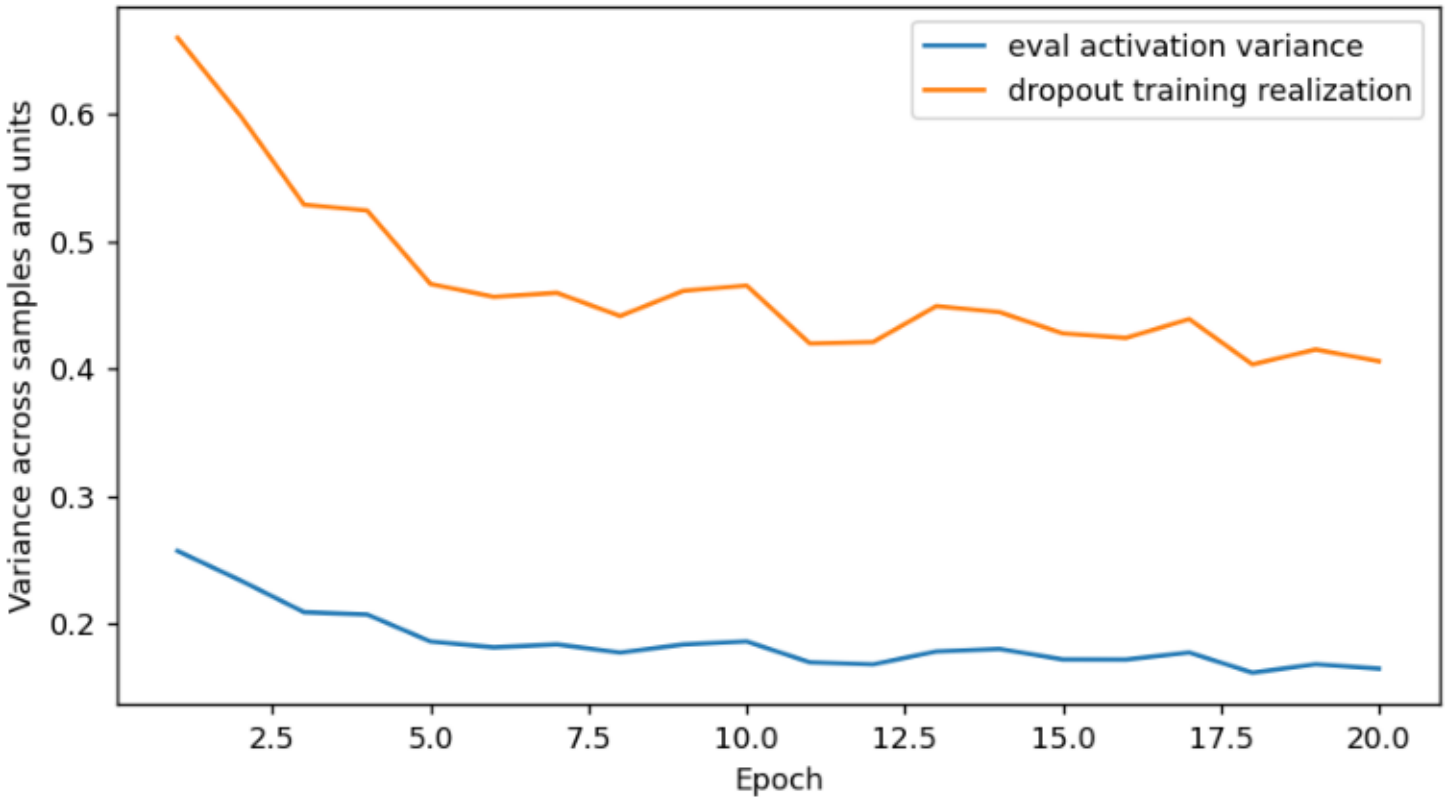
Lê Sơn Trường — Kết quả mục 5.6
validation_learning_curves

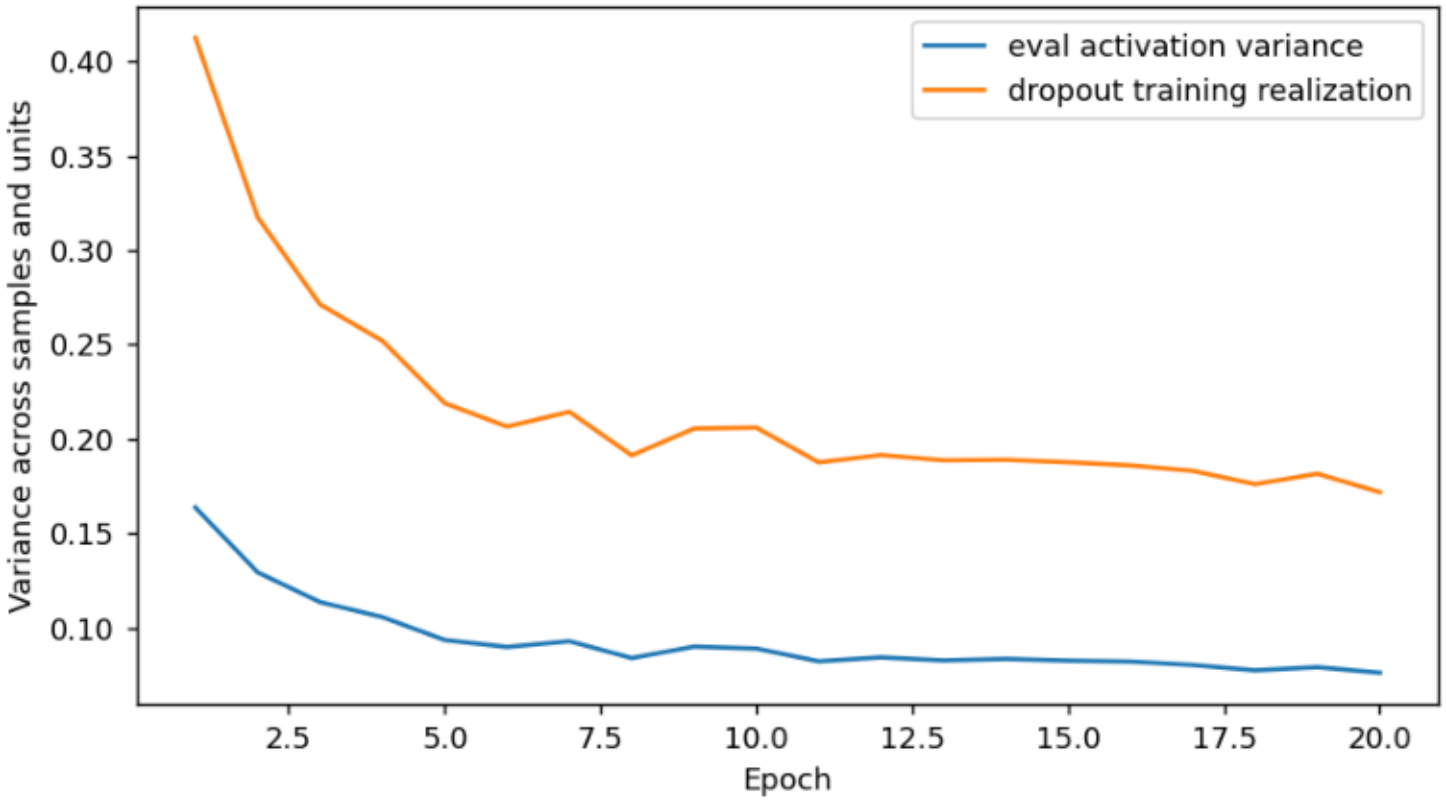


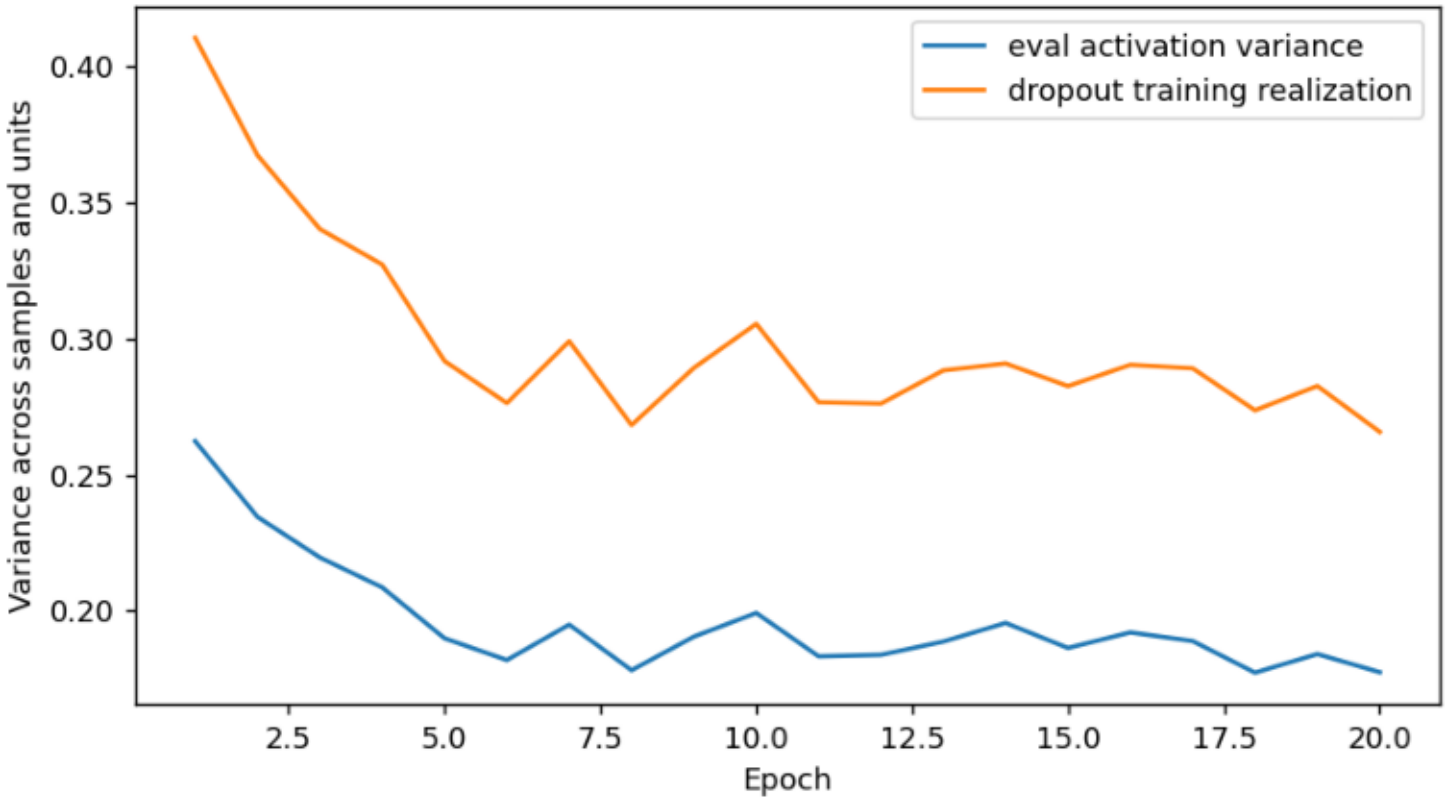

eval activation variance
dropout training realization
Variance across samples and units
0.28
0.26
0.24
0.22
0.20
0.18
0.16
0.14
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Epoch


eval activation variance
dropout training realization
Variance across samples and units
0.34
0.32
0.30
0.28
0.26
0.24
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Epoch


eval activation variance
dropout training realization
Variance across samples and units
0.25
0.20
0.15
0.10
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Epoch


eval activation variance
dropout training realization
Variance across samples and units
0.6
0.5
0.4
0.3
0.2
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Epoch


eval activation variance
dropout training realization
Variance across samples and units
0.40
0.35
0.30
0.25
0.20
0.15
0.10
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Epoch


eval activation variance
dropout training realization
Variance across samples and units
0.40
0.35
0.30
0.25
0.20
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Epoch

Lê Sơn Trường — Chương 5, mục 5.7

5.7 — Predicting House Prices on Kaggle (5 câu)

Câu 1

Đã xây pipeline tạo Le_Son_Truong_submission.csv, hai cột Id/SalePrice,
1.459 dòng, dự đoán giá dương bằng trung bình dự đoán giá của 5 mô hình fold.
Đã kiểm tra schema, ID, số dòng và số hữu hạn. Chưa nộp lên Kaggle và chưa có
điểm leaderboard; điểm 5-fold local không được ghi thành điểm Kaggle.
Bạn đăng nhập, chấp nhận quy tắc House Prices và upload CSV ở trang cuộc thi:
https://www.kaggle.com/competitions/house-prices-advanced-regression-techniques.
Sau khi có điểm, chụp màn hình và bổ sung vào hồ sơ. Đây là bước còn cần tài khoản cá nhân.

Câu 2

Không luôn nên mean-impute. Ví dụ giá trị thu nhập cao dễ bị người trả lời bỏ trống:
missingness phụ thuộc chính giá trị không quan sát (MNAR), mean từ người trả lời
bị thiên lệch. Với nhà không có garage, missing diện tích garage có thể nghĩa là
không tồn tại, không phải gara “trung bình”. Cần hiểu ngữ nghĩa, có missing indicator,
imputation có điều kiện hoặc mô hình phù hợp; mọi thống kê chỉ fit trên train fold.

Câu 3

Notebook dùng 5-fold cố định seed42 để so baseline linear SGD LR=.01,10 epoch
của sách với 100 epoch và weight decay. Báo cáo mean±std log RMSE;
sách có chỗ in log MSE, phải phân biệt MSE với căn của nó khi so sánh.
Chọn cấu hình theo CV, không dùng test labels. Lưới hữu hạn không bảo đảm tối ưu
toàn cục; repeated tuning có selection bias.

Câu 4

Thử MLP một tầng64 và hai tầng64→32, ReLU, Adam LR=.003,100 epoch,
Dropout .1 và decay tương ứng 1e−4/1e−3. Đây là mở rộng được câu hỏi yêu cầu,
không phải giả định sách mặc định dùng Adam/100 epoch.
So CV với baseline, chọn mô hình tốt nhất trong tập đo rồi ensemble 5 fold để
dự đoán. Mạng sâu hơn không nhất thiết tốt hơn dữ liệu bảng nhỏ.

Câu 5

Bỏ scaling làm các feature chênh lệch độ lớn khiến tối ưu kém điều kiện,
gradient bị biến có scale lớn chi phối; cùng LR có thể phân kỳ. Notebook chạy
baseline không scale ở fold1 và ghi trạng thái finite/diverged thực tế.
Scaling còn ảnh hưởng ý nghĩa regularization. Bản này fit mean/std và category
trong mỗi train fold rồi áp cho validation/test, thay vì dùng toàn bộ train+test
như cách đơn giản minh họa trong sách, để tránh leakage thống kê trong CV.

Ghi nhận chạy thật — mục 5.7

Selected config=mlp64_dropout; 5-fold log RMSE=0.26146; submission=Le_Son_Truong_submission.csv, 1459 rows.

Kaggle submission has NOT been uploaded. Leaderboard score is unknown; local CV score is not a Kaggle score. User must accept competition rules and upload via own account.

Kết luận thực nghiệm — mục 5.7

Nhận xét từ số đo: mlp64_dropout có 5-fold log RMSE 0.26146 ± 0.03495; baseline 10 epoch là 0.52599. Đây là CV local, chưa phải điểm Kaggle. CSV dự đoán đã tạo, còn thao tác nộp bằng tài khoản cá nhân.

Kết quả mục 5.7 — kfold_scores

name	fold	train_log_rmse	val_log_rmse
book_linear_10epochs	1	0.488287	0.470547
book_linear_10epochs	2	0.498995	0.530303
book_linear_10epochs	3	0.487374	0.559276
book_linear_10epochs	4	0.491663	0.537638
book_linear_10epochs	5	0.496217	0.532206
linear_100epochs	1	0.215584	0.244708
linear_100epochs	2	0.212746	0.29559
linear_100epochs	3	0.20622	0.303993
linear_100epochs	4	0.218239	0.259704
linear_100epochs	5	0.217092	0.258217
ridge_100epochs	1	0.225641	0.250249
ridge_100epochs	2	0.223995	0.302423
ridge_100epochs	3	0.21673	0.310119
ridge_100epochs	4	0.228211	0.268638
ridge_100epochs	5	0.227591	0.271574
mlp64_dropout	1	0.210168	0.231569
mlp64_dropout	2	0.202486	0.246994
mlp64_dropout	3	0.186838	0.26142
mlp64_dropout	4	0.280427	0.321066
mlp64_dropout	5	0.231756	0.246251
mlp64_32_dropout	1	0.377598	0.392142
mlp64_32_dropout	2	0.279945	0.33253
mlp64_32_dropout	3	0.318239	0.398185
mlp64_32_dropout	4	0.624649	0.636334
mlp64_32_dropout	5	0.300574	0.325887

Kết quả mục 5.7 — model_selection

name	hidden	lr	epochs	optimizer	decay	dropout	cv_log_rmse_mean	cv_log_rmse_std
book_linear_10epochs	()	0.01	10	sgd	0	0	0.525994	0.0330711
linear_100epochs	()	0.01	100	sgd	0	0	0.272442	0.0258121
ridge_100epochs	()	0.01	100	sgd	0.01	0	0.2806	0.024967
mlp64_dropout	(64,)	0.003	100	adam	0.0001	0.1	0.26146	0.0349531
mlp64_32_dropout	(64, 32)	0.003	100	adam	0.001	0.1	0.417016	0.127

Kết quả mục 5.7 — data_summary

train_rows	unlabeled_test_rows	raw_features	missing_cell_fraction
1460	1459	79	0.0678776

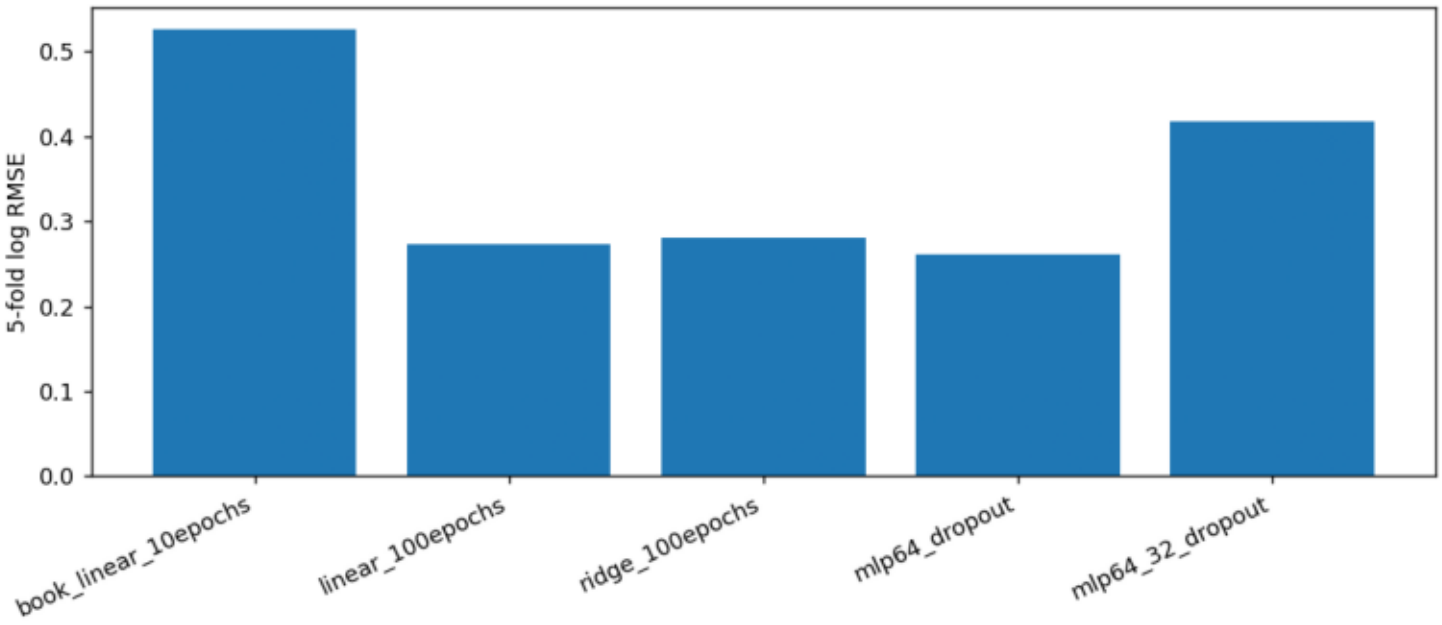
Kết quả mục 5.7 — without_standardization

standardize	fold	train_log_rmse	val_log_rmse	status
False	1	NaN	NaN	diverged

Kết quả mục 5.7 — submission_preview

Id	SalePrice
1461	117751
1462	92229.1
1463	175116
1464	194087
1465	208077
1466	166240
1467	176589
1468	154845
1469	202468
1470	114036

kfold_model_comparison



Lê Sơn Trường — Ôn tập Loss, backpropagation và MLP

Ôn tập Buổi 2 — Lê Sơn Trường

1. Loss là gì?

Loss đo mức sai của dự đoán, để mô hình biết cần điều chỉnh trọng số thế nào.

- Hồi quy: MSE hoặc half-MSE, ví dụ $L=(\hat{y}-y)^2/2$. Sai càng xa thì phạt càng mạnh.
- Phân loại: cross-entropy, $L=-\log p(\text{lớp đúng})$. Đoán sai mà rất tự tin bị phạt mạnh.
- Loss khác accuracy: accuracy chỉ đếm đúng/sai; loss còn xét xác suất tự tin.

Với cross-entropy, đưa logits thô vào nn.CrossEntropyLoss, không softmax
trước một lần nữa. Mục 4.1/4.4 giải thích ổn định số và gradient $p-y$.

2. Backpropagation hoạt động thế nào?

Forward: dữ liệu → Linear → activation → logits → loss.
Backward: đi ngược đồ thị, dùng chain rule, tính đạo hàm loss theo từng trọng số.
Gradient cho biết chiều làm loss tăng; optimizer thường bước theo chiều ngược lại.

Ví dụ $\hat{y}=wx+b$, $L=(\hat{y}-y)^2/2$:
$\partial L/\partial w=(\hat{y}-y)x$; $\partial L/\partial b=\hat{y}-y$.
loss.backward() tính gradient, chưa tự cập nhật trọng số.

3. Tối ưu hóa

SGD cập nhật $w \leftarrow w - \text{learning_rate} \times \text{gradient}$ trên minibatch.
Adam dùng các trung bình trượt của gradient và bình phương gradient để điều chỉnh bước.
Learning rate nhỏ có thể học chậm; quá lớn có thể dao động hoặc phân kỳ.
Không có optimizer/LR tốt nhất cho mọi dữ liệu và ngân sách epoch.

4. Quy trình huấn luyện MLP

1. Đọc, xử lý dữ liệu; phân biệt train, validation và test.
2. Chọn mạng, loss, optimizer, batch size và số epoch.
3. Với mỗi epoch, đưa từng batch train qua mạng; zero gradient → forward → loss
   → backward → optimizer.step.
4. Đánh giá validation bằng model.eval() và torch.no_grad(); không cập nhật từ validation.
5. Xem learning curve, chọn mô hình/siêu tham số theo validation.
6. Sau khi chốt, mới đánh giá trên test độc lập nếu có.

```
python
for epoch in range(epochs):
    model.train()
    for X, y in train_loader:
        optimizer.zero_grad()
        logits = model(X)
        loss = criterion(logits, y)
        loss.backward()
        optimizer.step()

    model.eval()
    with torch.no_grad():
        # Tính validation loss/accuracy, cân theo số mẫu.
        pass
```

Một epoch là một lượt qua toàn bộ train, không phải một batch hay một bước update.

Dropout hoạt động lúc train và tắt lúc eval; no_grad tắt xây graph, không thay thế eval.
Trong ví dụ Fashion-MNIST theo sách, test được dùng như validation, nên kết quả
tuning ở bài này không được quảng bá như đánh giá trên test còn nguyên độc lập.

5. Các câu có thể được hỏi khi trả bài

- Vì sao cần activation? Nếu chỉ ghép Linear thì toàn mạng vẫn là affine.
- Vì sao train tốt nhưng validation kém? Mạng có thể ghi nhớ nhiễu/chi tiết train (overfitting).
- Vì sao zero init MLP không học? Neuron ẩn đối xứng; với ReLU, gradient có thể chết tại 0.
- Vì sao không reset gradient? PyTorch cộng dồn .grad; bỏ reset khiến update không còn đúng minibatch SGD thông thường.
- Dropout có luôn tốt? Không; phụ thuộc mức nhiễu, kiến trúc và thời gian học. Báo đúng kết quả đã chạy.
- Vì sao cần validation? Để chọn mô hình mà không tối ưu trực tiếp trên test độc lập.
- Vì sao chuẩn hóa House Prices? Tránh chênh lệch scale gây tối ưu kém điều kiện và regularization lệch.